JPEG-Like Image Compression Library

The complete guide

what it is, why it works, how the code does it,
how to run it on bwUniCluster, and how to talk about it

Written for: Syed Akram Ameer Abbas
Project: "JPEG-Like Image Compression Library" (C++, Python, NumPy), project 2 on the resume
Level of the implementation: what a solid M.Sc. graduate in an embedded / computing programme would build: readable, tested, measured, and honest about its limits
Target: bwUniCluster 3.0 (KIT), GNU toolchain, Slurm
Code: the `jpeg_codec/` directory delivered together with this PDF

Everything printed in this guide as a number, table, figure or listing was produced by the code in the project: no value was typed in by hand. Where a number is *yours to measure* (the cluster benchmarks), the guide says so.

Contents

1 The big picture **5**

1.1 What you built, in three sentences . . . 5
1.2 The pipeline . . . 5
1.3 What the quality slider does to a picture . . . 5
1.4 Where everything lives . . . 5
1.5 What this project is, and what it is not . . . 6
1.6 How to use this guide . . . 6

2 Step 0: pixels, tiles, and a shift **8**

2.1 Pixels and why raw images are big . . . 8
2.2 Why tiles? . . . 8
2.3 Padding when the size is not a multiple of 8 . . . 8
2.4 The level shift: subtract 128 . . . 9
2.5 The numbers we will follow through the whole guide . . . 9

3 The DCT: recipes for pictures **10**

3.1 The intuition . . . 10
3.2 The formula, one dimension first . . . 11
3.3 Two dimensions: do rows, then columns . . . 11
3.3.1 Why the inverse is just the transpose . . . 12
3.3.2 A sanity check you can do by hand . . . 12
3.4 Why the DCT and not the Fourier transform? . . . 12
3.5 The code . . . 12

4 Quantization: the one place information is lost **14**

4.1 The intuition . . . 14
4.2 The step-size table . . . 14
4.3 Turning the “quality” knob . . . 14
4.4 Where exactly is the loss? . . . 15
4.5 Rounding ties and a subtle bug I found . . . 15

5 Zig-zag, DC prediction and run-length coding **17**

5.1 Zig-zag: sort by importance . . . 17
5.2 DC prediction: send differences . . . 18

5.3 AC coefficients: (run, size) pairs . . . 18
5.3.1 Amplitude bits: how negative numbers are stored . . . 18
5.3.2 Two special symbols . . . 19
5.4 Our example tile, token by token . . . 20

6 Huffman coding: short codes for common things 21
6.1 The intuition . . . 21
6.2 Building the code: merge the two rarest . . . 21
6.2.1 A textbook example (every number below is computed by the code) . . . 21
6.3 The implementation details that matter . . . 22
6.3.1 Only the lengths are transmitted: canonical codes . . . 22
6.3.2 What goes in the file . . . 22
6.3.3 Decoding . . . 22
6.3.4 Three engineering details . . . 23
6.4 Why two tables, and why the amplitude bits are not Huffman coded . . . 23

7 The file format and the decoder 25
7.1 The `JLC1` file format . . . 25
7.2 The encoder, end to end . . . 25
7.3 The decoder: run it backwards . . . 26
7.3.1 Malformed input . . . 27
7.4 Using the library from your own code . . . 27

8 Measuring compression: rate and distortion 28
8.1 The four numbers . . . 28
8.2 The rate–distortion trade-off . . . 28
8.2.1 The cameraman in numbers . . . 28
8.2.2 What kind of picture compresses well? . . . 29
8.3 Sanity check against libjpeg . . . 29

9 Rounding, ranges and numerical error 31
9.1 How big can a coefficient get? . . . 31
9.2 Where does the reconstruction error come from, and how big is it? . . . 31
9.3 Quality 100: pure rounding error . . . 32
9.4 Floating-point precision of the transform . . . 32
9.5 Rounding ties: why NumPy and C++ disagreed . . . 33

10 How we know it is correct 34

10.1 Layer 1: what the unit tests prove . . . 34
10.2 Layer 3: the NumPy reference model . . . 35

11 Performance: where the time goes **37**
11.1 How the code measures itself . . . 37
11.2 The profile . . . 37
11.3 Profile → hypothesis → experiment → fix . . . 38
11.4 How to profile on the cluster . . . 39
11.5 Fill in your cluster results . . . 39
11.6 How I would make it faster (and what to say about each idea) . . . 40
11.7 The GPU connection (relevant to the NVIDIA role) . . . 40

12 Running everything on your own machine **41**
12.1 Build and test (2 minutes) . . . 41
12.2 Use the codec . . . 41
12.3 Reproduce every result in this guide . . . 41
12.4 Rebuild this PDF . . . 41
12.5 Exercises that make the project yours . . . 41

13 Running on bwUniCluster 3.0: step by step **43**
13.1 The workflow in one picture . . . 43
13.2 One-time setup: account and login . . . 43
13.3 Where to put files . . . 44
13.4 Step 1: copy the project to the cluster . . . 44
13.5 Step 2: look around . . . 44
13.6 The queues you will use . . . 44
13.7 Step 3: build and test on a compute node . . . 45
13.8 Step 4: the benchmark jobs . . . 45
13.8.1 What the #SBATCH lines mean . . . 46
13.8.2 Benchmark hygiene (what makes a number trustworthy) . . . 46
13.9 Step 5: profile . . . 46
13.10 Step 6: bring the results home and plot . . . 46
13.11 When something goes wrong . . . 47
13.12 How to describe it truthfully . . . 47
13.13 Sources . . . 47

14 The resume bullets, one by one **48**
14.1 Bullet 1 . . . 48

14.2 Bullet 2 . . . 48
14.3 Bullet 3 . . . 49
14.4 Bullet 4 . . . 49
14.5 Bullet 5 . . . 49
14.6 Resume wording to consider once your cluster numbers exist . . . 50

15 Questions you should expect, with answers **51**
15.1 The project . . . 51
15.2 Measurement and numerics . . . 52
15.3 Performance and parallelism . . . 53
15.4 Scope, honesty and what next . . . 54
15.5 Beyond the project (an exploratory call) . . . 54

16 Drills: do these without the guide **56**
16.1 Hand calculations . . . 56
16.2 Answers . . . 56
16.3 Explain aloud (2 minutes each, no notes) . . . 57
16.4 An eight-day plan . . . 57

A Glossary and cheat sheet **58**
A.1 Terms in plain words . . . 58
A.2 Command cheat sheet . . . 59

Chapter 1

The big picture

1.1 What you built, in three sentences

You built a program that shrinks a grayscale picture to a small fraction of its size by throwing away the details a human eye barely notices, and a second program that rebuilds the picture from the small file. It does this with exactly the recipe of the JPEG standard: cut the picture into 8×8 tiles, re-express each tile as a mix of simple wave patterns (the *DCT*), round the mix coarsely (*quantization*, the only step that loses information), and then write the rounded numbers down in a compact shorthand (*zig-zag, run-length, Huffman*). You then measured how small the files get, how much quality is lost, how accurate the arithmetic is, and where the running time goes.

In plain words

Compression is possible because pictures are *predictable*: neighbouring pixels look alike, and our eyes are forgiving about fine detail. The whole codec is a machine for (1) turning "predictable" into "lots of zeros" and (2) writing lots of zeros very cheaply.

1.2 The pipeline

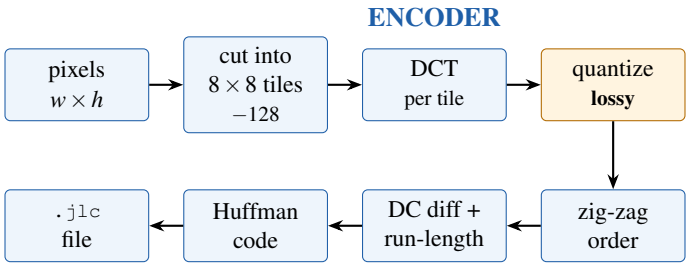


Figure 1.1: The encoder. The decoder is the same chain run backwards (Huffman decode → de-zig-zag → multiply by the step sizes → inverse DCT → add 128 and round). Only the orange box destroys information.

1.3 What the quality slider does to a picture

1.4 Where everything lives

File	What it does
`include/jlc/*.hpp`	Public headers, each with a comment block explaining the idea.
`src/dct.cpp`	The 8×8 DCT / inverse DCT (matrix form) plus a slow textbook version used only by tests.
`src/quant.cpp`, `quant.hpp`	Step-size table, the quality scaling rule, and the rounding rule.
`src/zigzag.cpp`	Builds the zig-zag scan order.

File	What it does
`src/rle.cpp`	Turns a block into tokens: DC difference, (run, size) pairs, ZRL, EOB.
`src/huffman.cpp`	Builds Huffman code lengths, canonical codes, table (de)serialization, decoding.
`include/jlc/bitio.hpp`	Writing and reading individual bits.
`src/codec.cpp`	`encode()` and `decode()`: the whole pipeline, with per-stage timers.
`src/metrics.cpp`	MSE, PSNR, maximum error.
`apps/jlc_cli.cpp`	Command line tool `jlc` (encode, decode, eval, sweep).
`apps/jlc_bench.cpp`	Benchmark: median per-stage times, CSV output.
`tests/test_main.cpp`	28 test groups, 30 000+ checks, no external framework.
`python/jlc_ref.py`	Independent NumPy re-implementation that reads and writes the same file format.
`python/crosscheck.py`	Proves C++ and NumPy produce byte-identical files.
`python/rd_sweep.py`	Rate–distortion sweep (also against libjpeg).
`python/numerics_study.py`	Rounding, range, and precision experiments.
`slurm/*.sbatch`	Job scripts for bwUniCluster.

1.5 What this project is, and what it is not

Being precise about scope makes you *more* credible in an interview, so here it is plainly.

- **It is** a complete, working, tested lossy codec for 8-bit grayscale images, using the same algorithmic ingredients as baseline JPEG, validated against libjpeg (same distortion at the same quality, file sizes within about 1%).
- **It is not** a drop-in JPEG file reader/writer. The file format ("JLC1") is our own: no JFIF headers, no marker bytes, one set of Huffman tables per image, no byte stuffing.
- **It does not** handle colour. Real JPEG converts RGB to YCbCr and subsamples the colour channels; this project is luminance only. Chapter 15 has the answer to "how would you add colour?".
- **It is single-threaded.** Parallelism (OpenMP over tiles, SIMD across tiles) is discussed as the next step in Chapter 11, not claimed as done.

Honesty rule for the interview

Say "JPEG-*like*" exactly as the resume does. Anyone who knows JPEG will respect a precise list of what you implemented and what you deliberately left out far more than a vague claim to have "built JPEG".

1.6 How to use this guide

1. Read Chapters 2–7 once, slowly, with the code open next to you. They explain the ideas in plain words first, then the maths, then the exact code.
2. Run everything yourself (Chapter 12), then on bwUniCluster (Chapter 13). Your cluster numbers replace the placeholders in Chapter 11.
3. Chapters 14–16 are the interview material. Do the drills aloud.

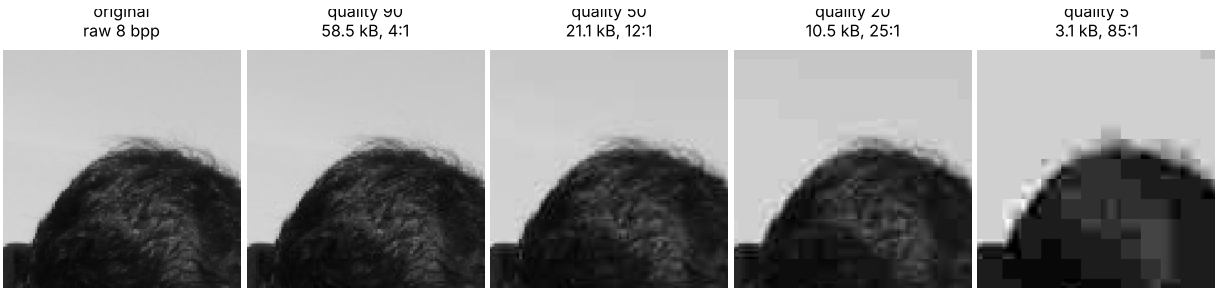


Figure 1.2: A 96×96 crop of the test photograph (the "cameraman") through the codec. File sizes are for the whole 512×512 image. At quality 5 you can see the 8×8 tiles: that "blocking" is the signature of JPEG-style compression.

Interview rules

The invitation e-mail states that using unapproved outside tools such as ChatGPT during the interview leads to disqualification. Treat this guide, the code, and any AI assistant as *preparation* material only: do not have them open or running during the call, and do not screen-share any of it unless the interviewer asks to see your own code. The goal of this guide is that you do not need it on the day.

Chapter 2

Step 0: pixels, tiles, and a shift

2.1 Pixels and why raw images are big

A grayscale image is a grid of numbers from 0 (black) to 255 (white). One number needs 8 bits, so a 512×512 image takes $512 \cdot 512 = 262{,}144$ bytes uncompressed. The PGM file format used by the project is nothing more than a tiny text header followed by exactly these bytes.

In plain words

Bits per pixel (bpp) is the number we will use all through the guide: "how many bits does the file spend on each pixel on average?" Raw is 8 bpp. The compressed cameraman at quality 50 costs about 0.64 bpp.

2.2 Why tiles?

The next step (the DCT) asks "which smooth or wiggly patterns is this piece of picture made of?" If we asked that for the whole picture at once, one fine detail in a corner would change *every* number. By cutting the picture into small independent tiles of 8×8 pixels, three things happen:

- **Local adaptation.** A flat sky tile needs almost no numbers; a tile containing an edge needs more. Each tile pays for what it contains.
- **Cheap maths.** An 8×8 transform is tiny (1024 multiply-adds, Chapter 3), fits in registers/L1 cache, and all tiles can be processed independently, which is why JPEG parallelizes so nicely on GPUs.
- **A price: blocking.** Tiles are compressed independently, so at very low quality their borders disagree and you see the grid (right side of the gallery figure). 8 is a historic compromise between "bigger tiles compress better" and "bigger tiles cost more and blur edges".

2.3 Padding when the size is not a multiple of 8

A 37×53 image does not divide into 8×8 tiles. The codec pads the image on the right and bottom by *repeating the last column and row*. It stores the true width and height in the file header, and the decoder crops the padding away.

In plain words

Why repeat edge pixels instead of padding with zeros? Zeros would create an artificial sharp edge (a bright picture suddenly dropping to black) in the border tiles. Sharp edges need many DCT coefficients, so zero padding would waste bits. Repeating the edge keeps the border tile smooth.

In the code this is two `std::min` calls inside `gatherBlock` in `src/codec.cpp`: the pixel read is clamped to the last valid row and column.

2.4 The level shift: subtract 128

Pixel values $0 \ldots 255$ are all positive, centred around 128. The DCT's first output (the "DC" coefficient, the tile's average brightness) would then always be large and positive. Subtracting 128 first moves values into $[-128, 127]$, centred on zero.

- It does not change the information; the decoder simply adds 128 back.
- It keeps the DC coefficient small and symmetric, which makes it cheaper to code, and it gives us a clean bound used later: $|\text{coefficient}| \leq 1024$ (Chapter 9).

2.5 The numbers we will follow through the whole guide

To make the next chapters concrete, we follow *one real tile* of the cameraman image through every stage: tile column 25, tile row 7. (It was chosen by a script because it contains a bit of texture, not just flat sky.)

$$\underbrace{\begin{pmatrix} 205 & 205 & 204 & 204 & 205 & 204 & 204 & 205 \\ 205 & 205 & 205 & 205 & 205 & 205 & 204 & 204 \\ 205 & 205 & 205 & 204 & 205 & 204 & 203 & 204 \\ 205 & 204 & 205 & 203 & 200 & 202 & 202 & 204 \\ 205 & 205 & 202 & 201 & 204 & 204 & 205 & 205 \\ 204 & 201 & 203 & 201 & 199 & 202 & 202 & 204 \\ 190 & 190 & 190 & 190 & 189 & 187 & 191 & 194 \\ 172 & 164 & 156 & 154 & 156 & 158 & 167 & 169 \end{pmatrix}}_{\text{original pixels 0...255}} \xrightarrow{-128} \underbrace{\begin{pmatrix} 77 & 77 & 76 & 76 & 77 & 76 & 76 & 77 \\ 77 & 77 & 77 & 77 & 77 & 77 & 76 & 76 \\ 77 & 77 & 77 & 76 & 77 & 76 & 75 & 76 \\ 77 & 76 & 77 & 75 & 72 & 74 & 74 & 76 \\ 77 & 77 & 74 & 73 & 76 & 76 & 77 & 77 \\ 76 & 73 & 75 & 73 & 71 & 74 & 74 & 76 \\ 62 & 62 & 62 & 62 & 61 & 59 & 63 & 66 \\ 44 & 36 & 28 & 26 & 28 & 30 & 39 & 41 \end{pmatrix}}_{\text{level-shifted}}$$

Chapter 3

The DCT: recipes for pictures

3.1 The intuition

Think of a sound. A chord on a piano *looks* like a complicated wiggly line, but it is really three or four pure notes added together. Writing down "note A at volume 3, note C at volume 1" is much shorter than writing down thousands of samples of the wiggly line.

Pictures work the same way. Any 8×8 tile can be built by adding up 64 fixed *pattern tiles*, each scaled by some amount. The 64 patterns are shown below: the top-left is a flat grey (no variation), moving right adds more vertical stripes, moving down adds more horizontal stripes, and the bottom-right is a fine checkerboard.

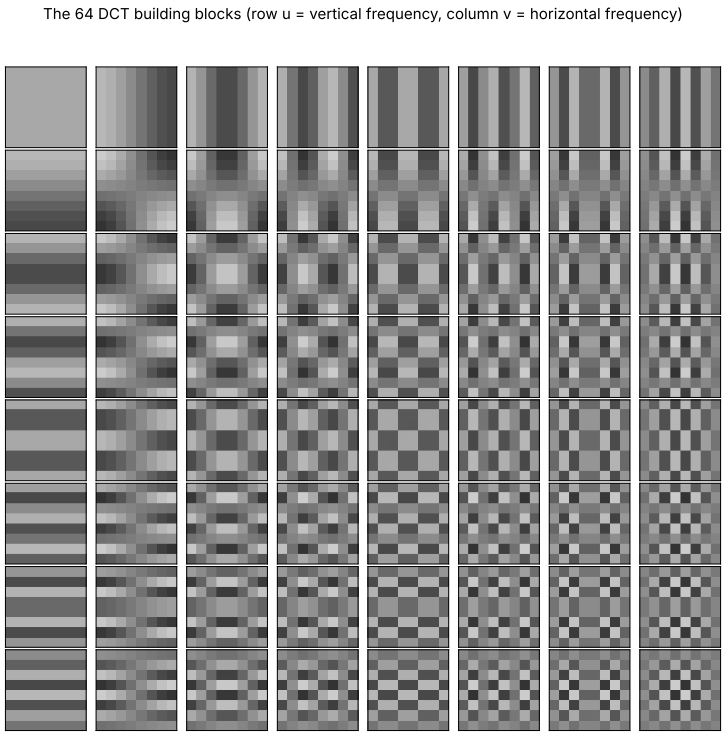


Figure 3.1: The 64 building blocks of the 8×8 DCT. Any tile is a weighted sum of these. Weights near the top-left (smooth patterns) are usually big, weights near the bottom-right (fine detail) are usually tiny.

The **DCT** (Discrete Cosine Transform) is the machine that answers "how much of each pattern is in this tile?" It outputs 64 numbers, one per pattern, called *coefficients*. The inverse DCT adds the patterns back up and returns the tile. Nothing is lost in this step; it is just a change of description.

In plain words

Why bother? Because for natural pictures, after the change of description, most of the 64 numbers are close to zero. The energy is *compacted* into a few coefficients. The next step can then round the small ones to exactly zero without anyone noticing.

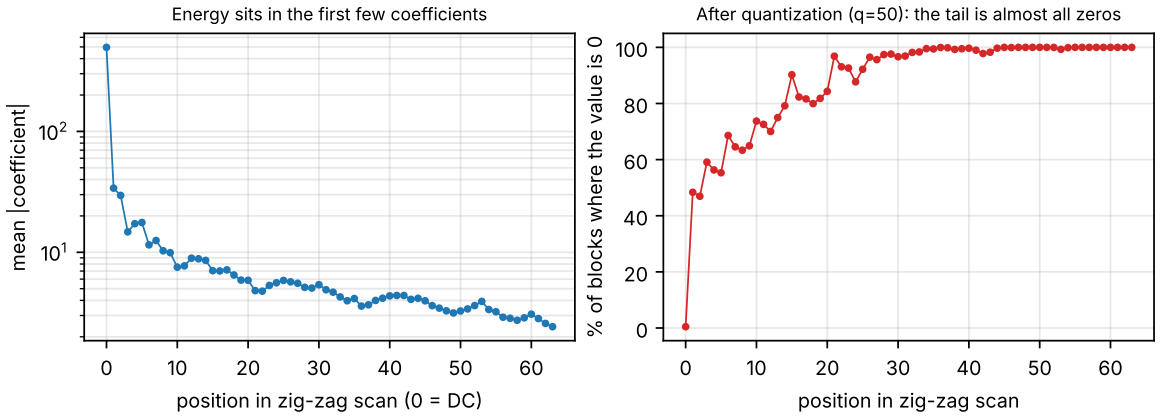


Figure 3.2: Left: average size of each coefficient (in zig-zag order, Chapter 5) over all tiles of the cameraman: the first few dominate by orders of magnitude. Right: after quantization at quality 50, the share of tiles in which each position is exactly zero.

3.2 The formula, one dimension first

Take 8 numbers $x_0, \dots, x_7$ (one row of a tile). The DCT produces 8 coefficients

$$X_u \;=\; \alpha(u) \sum_{x=0}^{7} x_x \cos\!\left(\frac{(2x+1)\,u\,\pi}{16}\right), \qquad \alpha(0) = \sqrt{\tfrac{1}{8}}, \quad \alpha(u>0) = \sqrt{\tfrac{2}{8}} = \tfrac{1}{2}. \tag{3.1}$$

How to read it:

- u is the *frequency*: how many half-waves of a cosine fit across the 8 samples. $u = 0$ is a flat line, $u = 1$ is one slow half-wave from bright to dark, $u = 7$ wiggles up and down almost every sample.
- The sum multiplies the data by that cosine and adds it up. If the data looks like the cosine, the sum is large (the pattern is "present"); if not, positive and negative terms cancel and the sum is near zero.
- The factor $\alpha(u)$ is only a scaling constant that makes the transform *orthonormal*, explained below.
- The odd-looking $(2x+1)/16$ means each sample is evaluated at the *centre* of its pixel. This choice (the "type-II" DCT) is what makes JPEG's tiles behave well at their borders.

3.3 Two dimensions: do rows, then columns

For a tile X (an 8×8 array), the 2D DCT has frequency u vertically and v horizontally:

$$Y_{uv} \;=\; \alpha(u)\alpha(v) \sum_{x=0}^{7} \sum_{y=0}^{7} X_{xy} \cos\!\left(\tfrac{(2x+1)u\pi}{16}\right) \cos\!\left(\tfrac{(2y+1)v\pi}{16}\right). \tag{3.2}$$

This is 64 outputs, each a sum of 64 terms: 4096 multiply-adds. But the formula *factorizes*, so we can do a 1D DCT on every column, then a 1D DCT on every row. In matrix language, define the 8×8 matrix C with entries $C_{ux} = \alpha(u) \cos\left(\frac{(2x+1)u\pi}{16}\right)$. Then

$$\boxed{Y \;=\; CXC^{\top}} \qquad \text{and the inverse is} \qquad \boxed{X \;=\; C^{\top} Y C} \tag{3.3}$$

Each matrix product costs $8 \cdot 8 \cdot 8 = 512$ multiply-adds, two of them make **1024**, a $4\times$ saving over the double sum. (Dedicated fast algorithms such as AAN need only roughly 5 multiplications and 29 additions per 8-point transform; our code does not use them, on purpose, for clarity.)

3.3.1 Why the inverse is just the transpose

The rows of C are the 8 cosine patterns scaled by the α's so that each row has length 1 and any two different rows are perpendicular (dot product 0). That is exactly the statement

$$CC^\top = I \quad \Longrightarrow \quad C^{-1} = C^\top. \tag{3.4}$$

A matrix with this property is called *orthonormal* (a "rotation" in 8 dimensions). The test `dct_matrix_is_orthonormal` checks this numerically. Two useful consequences:

1. **Perfect reconstruction.** Apply C then $C^\top$ and you get the tile back (up to 10^{-13} floating-point noise).
2. **Energy conservation (Parseval).** Sum of squares is unchanged: $\sum Y_{uv}^2 = \sum X_{xy}^2$. Proof in one line, with $\|\cdot\|_F$ the sum of squares:

$$\|Y\|_F^2 = \operatorname{tr}\big(CXC^\top CX^\top C^\top\big) = \operatorname{tr}\big(CXX^\top C^\top\big) = \operatorname{tr}\big(XX^\top\big) = \|X\|_F^2.$$

   This is how we later predict pixel-domain error from coefficient-domain error, and why no single coefficient can exceed $\sqrt{64 \cdot 128^2} = 1024$.

3.3.2 A sanity check you can do by hand

For a perfectly flat tile of value a: only Y_{00} is non-zero and equals $8a$ (the test `dct_flat_block_has_only_dc_equal_to_8_times_value`). The DC coefficient is always *8 times the tile's average*. Our example tile after the DCT (rounded to integers for display):

$$\begin{pmatrix}
551 & 1 & 10 & 0 & 2 & 0 & 0 & 1 \\
78 & 2 & -11 & 0 & -1 & -2 & 1 & -1 \\
-59 & -1 & 6 & 1 & 2 & 0 & -2 & 1 \\
43 & -2 & -5 & -1 & -1 & 0 & 1 & 0 \\
-28 & 0 & 6 & 2 & 0 & 0 & -1 & 0 \\
13 & 1 & -3 & -4 & 1 & 0 & 1 & 0 \\
-7 & 1 & 3 & 2 & 0 & 0 & 1 & 0 \\
4 & -2 & -2 & 1 & 1 & -2 & -2 & 1
\end{pmatrix}$$

Look at the top-left entry (the DC, brightness of the whole tile) and how quickly the magnitudes fall away towards the bottom-right.

3.4 Why the DCT and not the Fourier transform?

- **Real numbers only.** The DFT produces complex numbers (two values per frequency). The DCT produces one real number per frequency.
- **No false edges at the borders.** The DFT implicitly repeats the tile periodically; a tile that goes from dark on the left to bright on the right becomes a sawtooth with a sudden jump, which smears energy over all frequencies. The DCT implicitly *mirrors* the tile at its borders (a triangle wave instead), so there is no jump, and the energy stays in a few low-frequency coefficients.

3.5 The code

The matrix is built once from the formula:

```
struct DctMatrix {
    double c[64];
    DctMatrix() {
        const double pi = std::acos(-1.0);
        for (int u = 0; u < kBlock; ++u) {
            const double alpha = (u == 0) ? std::sqrt(1.0 / kBlock) : std::sqrt(2.0 / kBlock);
            for (int x = 0; x < kBlock; ++x) {
                c[u * kBlock + x] = alpha * std::cos((2 * x + 1) * u * pi / (2.0 * kBlock));
            }
        }
    }
};

}  // namespace
```

and the forward transform is exactly the two matrix products $T = CX$, $Y = TC^{\top}$:

```
void forwardDCT8x8(const double* in, double* out) {
    const double* C = dctMatrix();
    double tmp[64];
    // Step 1: transform the columns:  T = C * X      T[u][y] = sum_x C[u][x] X[x][y]
    for (int u = 0; u < 8; ++u) {
        for (int y = 0; y < 8; ++y) {
            double s = 0.0;
            for (int x = 0; x < 8; ++x) s += C[u * 8 + x] * in[x * 8 + y];
            tmp[u * 8 + y] = s;
        }
    }
    // Step 2: transform the rows:  Y = T * C^T       Y[u][v] = sum_y T[u][y] C[v][y]
    for (int u = 0; u < 8; ++u) {
        for (int v = 0; v < 8; ++v) {
            double s = 0.0;
            for (int y = 0; y < 8; ++y) s += tmp[u * 8 + y] * C[v * 8 + y];
            out[u * 8 + v] = s;
        }
    }
}
```

In plain words

Reading the code. `in` and `out` are 64 numbers in row-major order (index $= 8 \cdot \text{row} + \text{col}$). The first double loop computes one column-transform output $T[u][y]$ as a dot product of row u of C with column y of the tile. The second does the same along rows. It uses `double`, so rounding noise is about 10^{-13}; Chapter 9 shows why that is far more than enough.

The function `forwardDCT8x8Naive` in the same file is the 4-fold sum straight from the textbook. It is slow and exists only so that the tests can prove the fast matrix version computes the same thing (agreement to 10^{-10} on 50 random tiles).

Interview soundbite

"The DCT is an orthonormal change of basis from pixel values to 64 cosine patterns. Orthonormal gives me two things for free: the inverse is the transpose, and squared error is preserved, so I can reason about distortion in either domain. I implemented it as $CXC^{\top}$, 1024 multiply-adds per tile, and validated it against the textbook double sum."

Chapter 4

Quantization: the one place information is lost

4.1 The intuition

Imagine you must report prices. Writing "€3.47" costs more characters than "€3.50", which costs more than "about €5". *Quantization* is deliberately reporting numbers at a coarser precision. You choose a step size Q and report only how many steps fit: $q = \text{round}(Y/Q)$. The decoder multiplies back, $\hat{Y} = q \cdot Q$, and the difference is the **quantization error**, at most $Q/2$.

Two things make this a good deal:

- **We choose a different step for every one of the 64 positions.** The eye is much less sensitive to errors in fine detail than in broad shading, so high-frequency positions get large steps (coarse rounding, lots of zeros) and low-frequency positions get small steps.
- **After the DCT most coefficients are small**, so with a big step they round to exactly 0. A zero costs almost nothing to store (Chapters 5–6).

4.2 The step-size table

The codec uses the standard JPEG luminance table ("quality 50"). Note the steps grow towards the bottom-right, where the fine patterns live:

$$Q_{50} = \begin{pmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{pmatrix}$$

4.3 Turning the "quality" knob

Users want one slider from 1 to 100, not 64 numbers. The rule (borrowed from libjpeg) scales the whole table:

$$s = \begin{cases} \lfloor 5000/\text{quality} \rfloor & \text{quality} < 50 \\ 200 - 2 \cdot \text{quality} & \text{quality} \geq 50 \end{cases} \qquad Q_i = \text{clamp}\left(\left\lfloor \frac{Q_i^{\text{base}} \cdot s + 50}{100} \right\rfloor, 1, 255 \right). \tag{4.1}$$

- Quality 50: $s = 100$, so $Q = Q^{\text{base}}$ (unchanged).
- Quality 100: $s = 0$, every entry clamps to 1, the finest possible rounding.
- Quality 10: $s = 500$, every step is $5\times$ larger, almost everything becomes zero.

- The "+50" inside the floor implements rounding to the nearest integer using only integer arithmetic.

```
QTable makeQuantTable(int quality) {
    if (quality < 1) quality = 1;
    if (quality > 100) quality = 100;
    const int scale = (quality < 50) ? 5000 / quality : 200 - 2 * quality;
    QTable q{};
    const QTable& base = baseLuminanceTable();
    for (int i = 0; i < 64; ++i) {
        int v = (base[i] * scale + 50) / 100;
        if (v < 1) v = 1;
        if (v > 255) v = 255;
        q[i] = v;
    }
    return q;
}
```

4.4 Where exactly is the loss?

Only here, plus two tiny effects at the very end of decoding: rounding the reconstructed numbers to whole pixel values, and clamping them into $[0, 255]$. Everything else (DCT, zig-zag, run-length, Huffman) is exactly reversible. The test `entropy_stage_is_exactly_lossless` proves that last claim: after any quantization, the decoded coefficient array equals the encoder's array bit for bit.

For our example tile at quality 50, quantization divides each coefficient by the table entry above and rounds:

$$\begin{pmatrix} 34 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 6 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ -4 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -2 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Out of 64 numbers, almost everything is now exactly zero. This is the whole trick. (On the full cameraman at quality 50, 89 % of all AC coefficients are zero, and on average a tile keeps only 7.7 non-zero values out of 64.)

4.5 Rounding ties and a subtle bug I found

"Round to nearest" has an ambiguity at exactly $x.5$. We use round-half-up, $\lfloor x + 0.5 \rfloor$. But in floating point, a value that is mathematically exactly 0.5 can come out as 0.49999999999999956 in one program and 0.50000000000000089 in another, depending on the order in which the 64 products are added. NumPy and the C++ loop add in different orders, so they disagreed on a handful of coefficients, and the two files were not byte-identical.

The fix is a deterministic tie-break: add a tiny $\varepsilon = 10^{-9}$ before flooring.

```
constexpr double kTieEpsilon = 1e-9;
inline double roundHalfUp(double x) { return std::floor(x + 0.5 + kTieEpsilon); }

// Quantize one coefficient: round(c / q).
inline int quantizeValue(double coefficient, int q) {
    return static_cast<int>(roundHalfUp(coefficient / q));
```

```
}
```

Chapter 9 explains why 10^{-9} is safe and shows how common near-ties are.

Interview soundbite

"Quantization is the only lossy operation: divide each DCT coefficient by a per-frequency step from the standard luminance table, scaled by the quality slider with libjpeg's rule, and round. During cross-validation against a NumPy model I found that exact rounding ties were broken differently because of floating-point summation order, and I fixed it with an explicit deterministic tie-break so both implementations produce byte-identical files."

Chapter 5

Zig-zag, DC prediction and run-length coding

After quantization each tile is 64 small integers, mostly zeros. This chapter turns them into a short list of "events" that the Huffman coder (next chapter) can write compactly.

5.1 Zig-zag: sort by importance

If we read the 8×8 tile row by row, the non-zero values are scattered among the zeros. If instead we read it along the anti-diagonals from the top-left corner outward, we visit coefficients roughly from *lowest to highest frequency*. Since high frequencies are mostly zero, the zeros bunch up into one long run at the end.

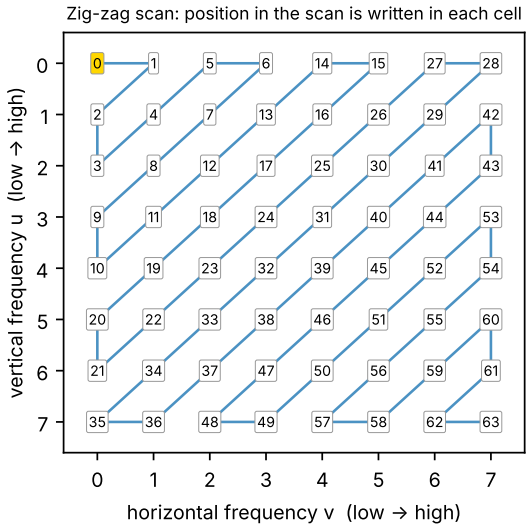


Figure 5.1: The zig-zag scan. Position 0 (gold) is the DC coefficient, 63 is the highest frequency. After quantization the tail of the scan is almost entirely zero.

The code builds the order by walking the 15 anti-diagonals (row + col = s), alternating direction:

```
std::array<int, 64> buildZigzag() {
    std::array<int, 64> order{};
    int k = 0;
    // Walk the 15 anti-diagonals (row + col = s). Odd diagonals run top-right to
    // bottom-left, even diagonals run bottom-left to top-right.
    for (int s = 0; s <= 14; ++s) {
        const int lo = (s > 7) ? s - 7 : 0;  // smallest row on this diagonal
        const int hi = (s < 7) ? s : 7;      // largest row on this diagonal
        if (s % 2 == 1) {
            for (int row = lo; row <= hi; ++row) order[k++] = row * 8 + (s - row);
        } else {
            for (int row = hi; row >= lo; --row) order[k++] = row * 8 + (s - row);
        }
    }
```

```
    return order;
}
```

The test `zigzag_is_permutation_and_has_jpeg_prefix` checks that every index 0..63 appears exactly once and that the first sixteen entries equal the well-known JPEG sequence $0, 1, 8, 16, 9, 2, 3, 10, 17, 24, \ldots$

For our example tile the 64 numbers in zig-zag order are

$$[34,\ 0,\ 6,\ -4,\ 0,\ 1,\ 0,\ -1,\ 0,\ 3,\ -2,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 0,\ 0,\ 0,\ \ldots]$$

(the trailing zeros are all zero). Now it is obvious what the next step does: describe this list briefly.

5.2 DC prediction: send differences

Position 0 is the *DC coefficient*, the tile's average brightness. Neighbouring tiles usually have similar averages (sky next to sky). So instead of the DC value we send its difference to the previous tile's DC. This is called **DPCM** (differential pulse-code modulation).

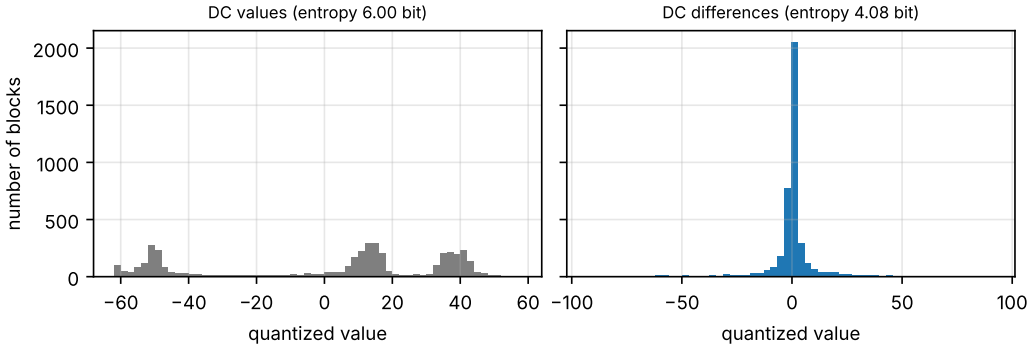


Figure 5.2: Quantized DC values of the cameraman (left) are spread widely; their differences from the previous tile (right) cluster around zero. The entropy, a lower bound on bits per value for a simple code, drops from 6.00 to 4.08 bits.

5.3 AC coefficients: (run, size) pairs

For the 63 remaining values we do not list the zeros one by one. We list only the non-zero values and say how many zeros came before each. Each non-zero value is described by two things:

- **run**: the number of zeros since the last non-zero value (0–15),
- **size** ("category"): the number of bits needed for its magnitude. Value $0 \to$ category 0, $\pm 1 \to 1$, $\pm 2, \pm 3 \to 2$, $\pm 4 .. \pm 7 \to 3$, and so on. (Code: `category()`.)

The pair $(\text{run}, \text{size})$ packs into one byte, $(\text{run} \ll 4) \,|\, \text{size}$, and *that byte* is the symbol that gets a Huffman code. After the code come `size` raw bits that say exactly which value in the category it is.

5.3.1 Amplitude bits: how negative numbers are stored

In category 3 the possible magnitudes are 4..7, so the sign and magnitude together have 8 possibilities, and 3 bits suffice. JPEG's trick: positive values are written as themselves; a negative value v is written as $v + (2^{\text{size}} - 1)$, i.e. the bitwise complement of its magnitude. For size 3: $4, 5, 6, 7 \to 100, 101, 110, 111$ and

$-4, -5, -6, -7 \to 011, 010, 001, 000$. The first bit of the amplitude therefore tells the sign (1 = positive). The decoder reverses this in `extendAmplitude`.

5.3.2 Two special symbols

EOB, end of block (`0x00`). Sent as soon as all remaining coefficients are zero. This is the big win: a typical tile ends after a handful of values, and the whole zero tail costs one short code (3 bits in our example).

ZRL, zero run length (`0xF0`). The run field has only 4 bits (0..15). A stretch of 16 or more zeros is broken into one ZRL per 16 zeros, then a normal pair for the rest. If the last coefficient (position 63) is non-zero there are no trailing zeros and no EOB.

```
int category(int v) {
    int a = std::abs(v);
    int n = 0;
    while (a != 0) {
        ++n;
        a >>= 1;
    }
    return n;
}

uint16_t amplitudeBits(int v, int size) {
    if (size == 0) return 0;
    const int mask = (1 << size) - 1;
    return static_cast<uint16_t>((v >= 0 ? v : v + mask) & mask);
}

int extendAmplitude(uint32_t bits, int size) {
    if (size == 0) return 0;
    // If the top bit is set the value is positive; otherwise it was stored as v + 2^size - 1.
    if (bits >= (1u << (size - 1))) return static_cast<int>(bits);
    return static_cast<int>(bits) - ((1 << size) - 1);
}

void tokenizeBlock(const int16_t* zz, int& prevDC, std::vector<Token>& out) {
    // --- DC: code the difference to the previous block (DPCM).
    const int diff = zz[0] - prevDC;
    prevDC = zz[0];
    const int dcSize = category(diff);
    out.push_back(Token{0, static_cast<uint8_t>(dcSize), static_cast<uint8_t>(dcSize),
                        amplitudeBits(diff, dcSize)});

    // --- AC: run-length code the zeros between non-zero coefficients.
    int run = 0;
    for (int k = 1; k < 64; ++k) {
        const int v = zz[k];
        if (v == 0) {
            ++run;
            continue;
        }
        while (run >= 16) {  // a run of 16 zeros is emitted as its own symbol
            out.push_back(Token{1, kZRL, 0, 0});
            run -= 16;
        }
        const int size = category(v);
        out.push_back(Token{1, static_cast<uint8_t>((run << 4) | size), static_cast<uint8_t>(size),
                            amplitudeBits(v, size)});
        run = 0;
    }
```

```
    if (run > 0) out.push_back(Token{1, kEOB, 0, 0});  // trailing zeros: end of block
}
```

5.4 Our example tile, token by token

Previous tile's DC was 37. The tile in the example has quantized DC 34, so we send the difference $34 - 37 = -3$. The table shows every symbol sent for the tile, the actual Huffman codes the encoder used for the full cameraman at quality 50, and the bit cost:

	symbol	meaning	value	Huffman code	amplitude	bits
DC	`0x02`	DC difference, category 2	-3	`100`	`00`	5
AC	`0x13`	run 1, category 3	6	`1111000`	`110`	10
AC	`0x03`	run 0, category 3	-4	`1010`	`011`	7
AC	`0x11`	run 1, category 1	1	`100`	`1`	4
AC	`0x11`	run 1, category 1	-1	`100`	`0`	4
AC	`0x12`	run 1, category 2	3	`11001`	`11`	7
AC	`0x02`	run 0, category 2	-2	`011`	`01`	5
AC	`0x91`	run 9, category 1	1	`11110111`	`1`	9
AC	`0x00`	EOB (rest is zero)		`010`	–	3

The whole tile costs **54 bits** instead of $64 \cdot 8 = 512$ bits: a local ratio of about 9.5:1, on a tile with non-trivial detail.

In plain words

Check it yourself. The value 6 is in category 3, run 1 (one zero before it) $\Rightarrow$ symbol $(1 \ll 4)|3 =$ `0x13`, amplitude bits `110`. The value -4 is category 3, so it is stored as $-4 + 7 = 3 =$ `011`. After -2 there are 9 zeros, then a 1 $\Rightarrow$ symbol $(9 \ll 4)|1 =$ `0x91`. Then nothing but zeros, so EOB.

Interview soundbite

"After quantization a block is mostly zeros, so I read it in zig-zag order to group the zeros at the end, send the DC as a difference to the previous block, and code each non-zero AC value as a (run, size) symbol plus raw amplitude bits, with a special end-of-block symbol for the zero tail, exactly as baseline JPEG does."

Chapter 6

Huffman coding: short codes for common things

6.1 The intuition

In Morse code, the most common letter in English, E, is a single dot, while rare letters such as Q take four symbols. The average message becomes much shorter than if every letter had the same length. **Huffman coding** builds the *best possible* such code for a given set of symbol frequencies, with one restriction: each symbol gets a whole number of bits, and no code is the beginning of another code (a *prefix-free* code), so the bit stream can be decoded without separators.

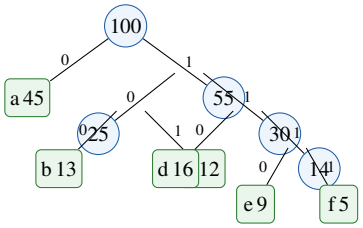
In our codec the "symbols" are the tokens from the last chapter: the 16 DC categories and the (run, size) bytes for AC values. A token such as "EOB" or "run 0, size 1" is extremely common, so it will get a 2–3 bit code; a rare token such as "run 14, size 7" may get 15 bits.

6.2 Building the code: merge the two rarest

1. Count how often each symbol occurs.
2. Take the two least frequent symbols, glue them under a new parent whose frequency is the sum.
3. Repeat with the new parent treated as a symbol, until one node (the root) remains.
4. The code of a symbol is the path from the root: left = 0, right = 1. Rare symbols hang deep in the tree and get long codes.

6.2.1 A textbook example (every number below is computed by the code)

symbol	frequency	length	code
`a`	45	1	`0`
`b`	13	3	`100`
`c`	12	3	`101`
`d`	16	3	`110`
`e`	9	4	`1110`
`f`	5	4	`1111`



Total: 224 bits for 100 symbols versus 300 bits with a fixed 3-bit code. The theoretical lower bound (entropy) is 2.22 bits per symbol and Huffman gives 2.24.

Merge order: f+e= 14; c+b= 25; 14+d= 30; 25+30= 55; a+55= 100. The test `huffman_textbook_example` asserts exactly these code lengths.

In plain words

Huffman is provably optimal among codes that give each symbol a whole number of bits: the average length L always satisfies $H \le L < H + 1$, where H is the entropy (the information-theoretic minimum). The test `huffman_kraft_equality_and_optimal_cost` checks this inequality on random distributions, and also that the code lengths satisfy $\sum 2^{-\ell_i} = 1$ (the code "uses up" the whole binary tree, with no wasted codewords).

6.3 The implementation details that matter

6.3.1 Only the lengths are transmitted: canonical codes

Sending the whole tree is wasteful. Instead, once the *lengths* are known, the actual codes are assigned by a fixed rule so the decoder can recompute them: sort symbols by (length, symbol value), give the first one the all-zero code, and give each next symbol the previous code $+1$, shifting left whenever the length increases. In the example above: b, c, d (length 3) get $100, 101, 110$; then the next code is 111, shifted left once for length 4: $1110, 1111$ for e, f.

```
    // Assign canonical codes: code(next) = (code(prev) + 1) << (len(next) - len(prev)).
    uint64_t code = 0;
    int prevLen = 0;
    int index = 0;
    for (int len = 1; len <= maxLength_; ++len) {
        code <<= (len - prevLen);
        prevLen = len;
        first_[len] = code;
        offset_[len] = index;
        for (int i = 0; i < count_[len]; ++i) {
            code_[sorted_[index]] = static_cast<uint32_t>(code);
            ++code;
            ++index;
        }
    }
}
```

6.3.2 What goes in the file

For each of the two tables (DC and AC), the file stores 32 bytes "how many codes have length 1, 2, …, 32" followed by the used symbols in canonical order. That is the same idea as JPEG's own DHT segment. In our tests the DC table takes 34–42 bytes and the AC table 33–94 bytes.

6.3.3 Decoding

The decoder reads one bit at a time, builds up a code, and checks whether it falls into the range of codes of the current length. Because canonical codes of one length are consecutive numbers, the check is just "is code $-$ first$[\ell] <$ count$[\ell]$".

```
int HuffTable::decodeSymbol(BitReader& br) const {
    uint64_t code = 0;
    for (int len = 1; len <= maxLength_; ++len) {
        code = (code << 1) | static_cast<uint64_t>(br.bit());
        if (count_[len] > 0 && code >= first_[len] && code < first_[len] + static_cast<uint64_t>(
    count_[len])) {
            return sorted_[offset_[len] + static_cast<int>(code - first_[len])];
```

```
        }
    }
    throw std::runtime_error("huffman: invalid code in bitstream");
}
```

(This bit-by-bit loop is simple and correct but not fast; real decoders use lookup tables. See Chapter 11.)

6.3.4 Three engineering details

Deterministic ties. When two nodes have equal frequency, which is merged first? The code always merges the pair with the smallest (frequency, smallest symbol in the subtree), so the resulting lengths are unique, and the NumPy reference makes the same choice. This is what lets the two implementations produce byte-identical files.

```
// Min-heap ordered by (frequency, smallest symbol).
auto later = [&nodes](int a, int b) {
    if (nodes[a].freq != nodes[b].freq) return nodes[a].freq > nodes[b].freq;
    return nodes[a].minSymbol > nodes[b].minSymbol;
};
std::priority_queue<int, std::vector<int>, decltype(later)> heap(later);
for (int i = 0; i < static_cast<int>(nodes.size()); ++i) heap.push(i);

while (heap.size() > 1) {
    const int a = heap.top();
    heap.pop();
    const int b = heap.top();
    heap.pop();
    Node parent{nodes[a].freq + nodes[b].freq, std::min(nodes[a].minSymbol, nodes[b].
minSymbol),
                a, b, -1};
    nodes.push_back(parent);
    heap.push(static_cast<int>(nodes.size()) - 1);
}
```

Length limit of 32. Pathological frequency distributions (Fibonacci-like) can create a code longer than the 32 bits the bit writer handles. If the tree gets too deep, all frequencies are halved (rounded up) and the tree rebuilt, which flattens it. The test `huffman_length_limit_is_enforced` builds the worst case with 45 Fibonacci frequencies and checks the limit, the Kraft inequality, and a round trip of every symbol.

A single used symbol. A flat image uses only one DC and one AC symbol. A tree with one node would give a zero-length code; the code instead assigns length 1 so something is still written and read. The tests cover it.

6.4 Why two tables, and why the amplitude bits are not Huffman coded

DC symbols (categories of differences) and AC symbols (run/size bytes) have completely different statistics, so each gets its own table. The raw amplitude bits that follow a symbol are nearly uniformly distributed (any value in the category is about equally likely), so Huffman coding them would save almost nothing but cost a huge table.

Interview soundbite

"I compute symbol frequencies in a first pass, build two Huffman tables per image with a deterministic tie-break, convert them to canonical codes so only the code lengths need to be stored, and write the stream in a second pass. Canonical codes also make decoding a small range check per bit length. I

verified optimality via the Kraft equality and the entropy bound in the unit tests.”

Chapter 7

The file format and the decoder

7.1 The JLC1 file format

magic JLC1	width u32	height u32	quality u8	DC Huffman table $32 + n_{DC}$ bytes	AC Huffman table $32 + n_{AC}$ bytes	entropy-coded blocks (bit stream, zero-padded)

All integers are little-endian. The payload contains the tiles in raster order (left to right, top to bottom). Each tile is: DC Huffman code, DC amplitude bits, then AC (code, amplitude) pairs until an EOB or the 63rd coefficient. There are no markers or byte stuffing (the real JPEG inserts `0x00` after every `0xFF` byte in the payload so that markers are recognizable; we have no markers, so we skip that).

In plain words

Header overhead. 13 bytes of header plus the two Huffman tables: 134 bytes in total for the 512×512 cameraman at quality 50, i.e. 0.6% of the file. For a small, flat image it matters: a mostly smooth, bright 128×128 corner crop of the cameraman at quality 50 is 218 bytes, of which 98 (45%) are header and tables. This is one reason a near-constant image can come out smaller than libjpeg's output, which has its own, larger, fixed overhead (Chapter 8).

7.2 The encoder, end to end

```
std::vector<uint8_t> encode(const Image& img, int quality, EncodeTimings* timings, bool fused) {
    const auto tStart = Clock::now();
    EncodeTimings local;
    EncodeTimings* tm = timings ? timings : &local;
    *tm = EncodeTimings{};

    QuantizedImage qi = analyze(img, quality, tm, fused);
    const size_t numBlocks = static_cast<size_t>(qi.blocksX) * qi.blocksY;

    // ---- Stage 3: DC prediction + run-length tokenization, count symbol frequencies.
    auto t0 = Clock::now();
    std::vector<Token> tokens;
    tokens.reserve(numBlocks * 8);
    int prevDC = 0;
    for (size_t b = 0; b < numBlocks; ++b) tokenizeBlock(&qi.zigzag[b * 64], prevDC, tokens);
    std::vector<uint64_t> dcFreq(kDcAlphabet, 0), acFreq(kAcAlphabet, 0);
    for (const Token& t : tokens) (t.isAC ? acFreq : dcFreq)[t.symbol]++;
    tm->scan_ms = msSince(t0);

    // ---- Stage 4: build the two Huffman tables from the symbol statistics.
    t0 = Clock::now();
    const HuffTable dcTable(huffmanCodeLengths(dcFreq));
    const HuffTable acTable(huffmanCodeLengths(acFreq));
    tm->huff_build_ms = msSince(t0);

    // ---- Stage 5: serialize header, tables and the entropy-coded payload.
    t0 = Clock::now();
```

```
    std::vector<uint8_t> out;
    out.reserve(kHeaderSize + 128 + tokens.size());
    out.insert(out.end(), kMagic, kMagic + 4);
    putU32(out, static_cast<uint32_t>(img.width));
    putU32(out, static_cast<uint32_t>(img.height));
    out.push_back(static_cast<uint8_t>(qi.quality));
    dcTable.serialize(out);
    acTable.serialize(out);
    {
        BitWriter bw(out);
        for (const Token& t : tokens) {
            const HuffTable& table = t.isAC ? acTable : dcTable;
            bw.put(table.code(t.symbol), table.length(t.symbol));
            bw.put(t.amp, t.nbits);
        }
        bw.flush();
    }
    tm->bitpack_ms = msSince(t0);
    tm->total_ms = msSince(tStart);
    return out;
}
```

In plain words

Why two passes? The Huffman tables depend on the symbol statistics of the *whole* image, so the encoder first generates all tokens and counts them (stage 3), builds the tables (stage 4), and only then writes bits (stage 5). The token list lives in memory in between. That is the price of a per-image optimal code; a decoder-friendly alternative is fixed standard tables (slightly bigger files, but one pass).

7.3 The decoder: run it backwards

1. Check the magic number, read width, height, quality. Reject impossible dimensions (so a corrupted header cannot make the program allocate terabytes).
2. Read the two Huffman tables.
3. For each tile: read the DC symbol, then `size` amplitude bits, and add to the running DC. Then loop reading AC symbols: EOB stops the tile, ZRL skips 16 positions, otherwise skip `run` positions and place the decoded value. Any run that would overflow position 63 is treated as an error.
4. De-zig-zag each tile and multiply by the step sizes of the stored quality (this is the approximation to the original DCT coefficients).
5. Inverse DCT, add 128, round to an integer, clamp into $[0, 255]$.
6. Crop the padding.

```
    BitReader br(data + pos, size - pos);
    int prevDC = 0;
    for (size_t b = 0; b < numBlocks; ++b) {
        int16_t* blk = &qi.zigzag[b * 64];
        // DC: category, then amplitude bits, then add to the previous DC.
        const int dcSize = dcTable.decodeSymbol(br);
        const int diff = extendAmplitude(br.bits(dcSize), dcSize);
        prevDC += diff;
        blk[0] = static_cast<int16_t>(prevDC);
        // AC: (run, size) symbols until EOB or the block is full.
        int k = 1;
        while (k < 64) {
```

```
            const int sym = acTable.decodeSymbol(br);
            const int run = sym >> 4;
            const int sz = sym & 15;
            if (sz == 0) {
                if (run == 15) {  // ZRL: sixteen zeros
                    k += 16;
                    if (k > 64) throw std::runtime_error("decode: run overflows block");
                    continue;
                }
                if (sym != kEOB) throw std::runtime_error("decode: invalid AC symbol");
                break;  // EOB
            }
            k += run;
            if (k > 63) throw std::runtime_error("decode: run overflows block");
            blk[k++] = static_cast<int16_t>(extendAmplitude(br.bits(sz), sz));
        }
    }
```

7.3.1 Malformed input

A decoder is a parser of untrusted data, so it must never crash or loop forever. The decoder throws an exception on a wrong magic number, truncated headers/tables/payload, invalid Huffman codes, and runs that overflow a tile. The test `malformed_streams_are_rejected_not_crashed` truncates a valid file at six places (each must throw) and flips random bits 200 times (the decoder may succeed with a damaged image or throw, but must never crash).

7.4 Using the library from your own code

```
//   Image img = readPGM("photo.pgm");
//   std::vector<uint8_t> bytes = jlc::encode(img, /*quality=*/50);
//   Image back = jlc::decode(bytes);
//
```

Chapter 8

Measuring compression: rate and distortion

8.1 The four numbers

Compression ratio $= \dfrac{\text{raw bytes}}{\text{compressed bytes}}$. "12:1" means the file is a twelfth of the original. Higher is smaller.

Bits per pixel (bpp) $= \dfrac{8 \cdot \text{compressed bytes}}{\text{number of pixels}}$. Raw is 8; this is the more convenient "rate" for plots because it does not depend on the picture size.

MSE (mean squared error) $= \dfrac{1}{N}\sum_i (\hat{p}_i - p_i)^2$ where p_i are original and $\hat{p}_i$ reconstructed pixels. Squaring punishes large errors and makes everything positive.

PSNR (peak signal-to-noise ratio) $= 10\log_{10}\dfrac{255^2}{\text{MSE}}$ decibels. It turns MSE into a number that grows as the picture gets better: 30 dB is visibly degraded, 40 dB is hard to tell apart, and every $+6$ dB halves the typical pixel error (because it multiplies $255^2/\text{MSE}$ by 4).

Limits of PSNR

PSNR is a pixel-by-pixel distance, not a model of human perception. Two images with the same PSNR can look very different (blurring versus blocky noise). Perception-based measures such as SSIM exist for this reason. PSNR remains the standard first number because it is simple, differentiable, and matches the maths of quantization noise.

8.2 The rate–distortion trade-off

Every lossy codec lives on a curve: a lower rate costs a higher distortion. The "quality" slider just moves along that curve. A *better codec* is one whose curve lies higher and further left (more PSNR for fewer bits).

8.2.1 The cameraman in numbers

quality	bytes	bpp	ratio	MSE	PSNR [dB]	max error
1	1 966	0.060	133.34	251.54	24.12	183
10	5 750	0.175	45.59	93.40	28.43	107
25	12 516	0.382	20.94	54.00	30.81	79
50	21 079	0.643	12.44	35.74	32.60	52
75	33 755	1.030	7.77	20.19	35.08	34
90	58 523	1.786	4.48	6.01	40.34	18
95	82 633	2.522	3.17	2.01	45.10	9
100	146 692	4.477	1.79	0.08	58.92	1

How to read it:

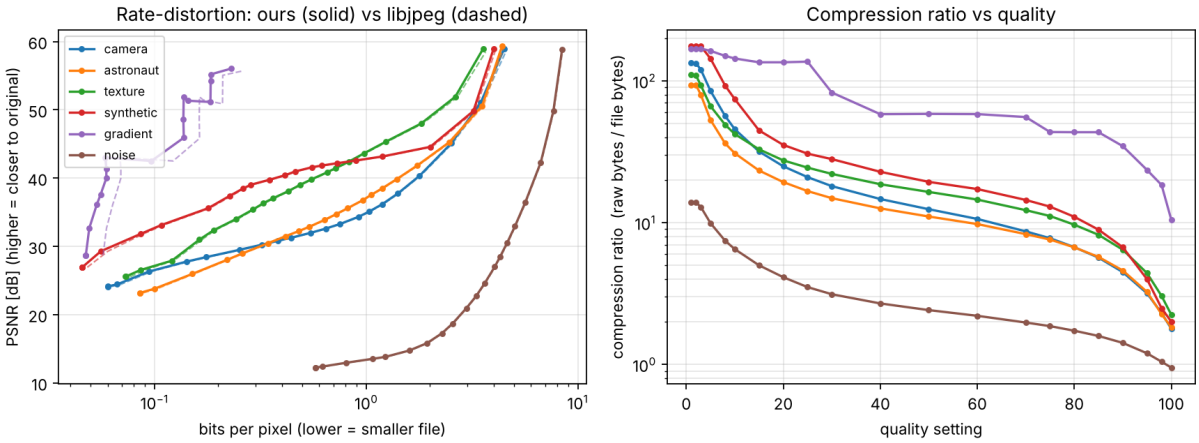


Figure 8.1: Left: PSNR versus bits per pixel for six test images (solid: this codec; dashed: libjpeg through Pillow, same quality values). Right: compression ratio versus quality setting.

- **Diminishing returns.** Going from quality 50 to 75 costs 1.6× the bytes and gains 2.5 dB; going from 95 to 100 multiplies the file by 1.8 for +14 dB that no human can see. The sweet spot for photographs is typically 75–90.
- **Quality 100 is not lossless.** The step is 1 everywhere, so the only remaining error is rounding of coefficients (Chapter 9): MSE 0.083, maximum error 1 grey level, and a file of 1.8:1.
- **Quality 1 still gives something recognizable** at about 130:1, but it is dominated by blocks.

8.2.2 What kind of picture compresses well?

smooth gradient	Almost all high-frequency coefficients are exactly zero already; at quality 50 the file is under 1.2 kB for 64 kB of pixels (more than 58:1).
photographs	Mixed smooth areas and edges: 8–20:1 is typical at quality 25–75.
white noise	Every pixel is independent, so no coefficient is small; quantization must throw away real information to save anything, PSNR collapses to about 23 dB at quality 50, and at quality 100 the file is *larger than the raw image* (ratio 0.95). Compression only works on structured data.

8.3 Sanity check against libjpeg

How do we know our numbers are right and not merely self-consistent? Run the *same* images through the reference implementation everybody uses, libjpeg (via the Pillow library), with the same quality values and optimized Huffman tables:

image	ours [B]	libjpeg [B]	difference	ours PSNR	libjpeg PSNR
camera	21 079	21 254	-0.8%	32.60	32.60
astronaut	23 730	23 936	-0.9%	34.75	34.75
texture	15 918	16 099	-1.1%	38.99	38.99
synthetic	13 519	13 674	-1.1%	40.41	40.41
gradient	1 121	1 332	-15.8%	48.56	48.56
noise	27 113	27 272	-0.6%	22.75	22.76



Figure 8.2: Where the error is, at quality 20. It is largest on sharp edges (camera, tripod, skyline) and spread evenly over the fine texture of the grass, where many small high-frequency coefficients were rounded to zero. Smooth sky is nearly error-free.

- **PSNR agrees to 0.01 dB** on every image. That is strong evidence that our DCT, tables, quality scaling and rounding match the standard, because the distortion depends only on those.
- **File sizes are within about 1% on natural images** (ours slightly smaller). Sizes depend on the entropy coder and on container overhead, which legitimately differ: our header is smaller than JFIF plus marker segments, and we do not need byte stuffing. On near-constant images the fixed overhead dominates and the difference is larger (gradient). We are *not* claiming to beat libjpeg; the comparison shows we are in the same league.
- At quality 100 ours reaches slightly higher PSNR than libjpeg (58.9 vs 58.5 dB on the cameraman). A plausible reason is that libjpeg's default DCT is a fixed-point integer algorithm, and ours uses `double`; I did not isolate this, so say "likely".

Interview soundbite

"I swept quality from 1 to 100 on six images and plotted PSNR against bits per pixel. At each quality the PSNR matches libjpeg to 0.01 dB and file sizes are within about 1% on natural images, which validates the transform and quantization stages independently of my own tests. White noise is the worst case; a smooth gradient is the best case."

Chapter 9

Rounding, ranges and numerical error

This chapter backs the resume line *"Investigated rounding, coefficient range, and numerical reconstruction errors introduced during compression"*. Each section asks a question, predicts the answer with a short calculation, and checks the prediction with an experiment (`python/numerics_study.py`, `tests/test_main.cpp`).

9.1 How big can a coefficient get?

Prediction. After the level shift, pixels lie in $[-128, 127]$, so the length of a tile as a vector of 64 numbers is at most $\sqrt{64 \cdot 128^2} = 1024$. The DCT is orthonormal, so it preserves length (Parseval, Chapter 3), and *every* single coefficient is at most as large as the length of the whole vector:

$$|Y_{uv}| \;\le\; \|Y\|_2 = \|X\|_2 \;\le\; 1024.$$

Consequences. Quantized values fit in a 16-bit integer (we store `int16_t`); a DC *difference* is at most $2 \cdot 1024 = 2048$, which has category 12; an AC value is at most 1024, category 11. The size field of a symbol has 4 bits (max 15), so it can never overflow. The test `coefficient_range_is_within_theory` checks the 1024 bound on random data, and in practice the black-tile extreme is reached exactly (the astronaut image contains a coefficient of -1024.0).

image	DC min	DC max	max |AC|	AC category
cameraman	−996	931	669	10
astronaut	−1024	1010	829	10
noise	−218	270	358	9

9.2 Where does the reconstruction error come from, and how big is it?

Model. Quantizing a coefficient with step Q makes an error that is roughly uniformly distributed in $[-Q/2, Q/2]$, with variance $Q^2/12$. By Parseval, errors in coefficients carry over unchanged to pixel errors, so the expected pixel MSE is the average of $Q_i^2/12$ over the 64 positions.

Experiment.

image	quality	measured MSE	model MSE	model / measured
camera	10	93.40	3719.21	39.8
camera	50	35.74	375.04	10.5
camera	90	6.01	14.99	2.5
camera	100	0.08	0.08	1.0
noise	10	2704.11	3719.21	1.4
noise	50	345.05	375.04	1.1
noise	90	14.81	14.99	1.0
noise	100	0.08	0.08	1.0

Reading. For white noise (every coefficient is large compared with Q) the model is close at every quality. For the real photograph it badly *over*-estimates at low quality ($40\times$ at quality 10). Why? The model assumes the coefficient is much larger than the step. But most high-frequency coefficients of a photograph are tiny; when $|Y| < Q/2$ the quantized value is 0 and the error is just $|Y|$, which is far smaller than $Q/2$. The model becomes exact as $Q \to 1$ (ratio 1.0 at quality 100). In other words: *the model is a worst-case-style bound, and the "dead zone" of coefficients quantized to zero is precisely what gives lossy compression its savings at a small cost.*

9.3 Quality 100: pure rounding error

With every $Q = 1$, each coefficient is off by at most 0.5 (variance $1/12$). By Parseval every pixel inherits error variance $1/12$ before the final rounding, a standard deviation of 0.289. Because the true pixel is an integer, the final rounding *removes* the error whenever $|e| < 0.5$, and leaves an error of ± 1 otherwise. If e is approximately Gaussian, the chance of $|e| > 0.5$ is

$$P = \operatorname{erfc}\left(\frac{0.5}{\sigma\sqrt{2}}\right)\Big|_{\sigma=\sqrt{1/12}} = 0.0833,$$

and since each wrong pixel contributes an error of exactly 1 the predicted MSE equals P. (That it lands within 0.1% of $1/12 = 0.0833$ is a numerical coincidence.) Measured:

image	measured MSE	predicted MSE	max error	pixels $\neq$ original
camera	0.0834	0.0833	1	8.34%
astronaut	0.0760	0.0833	2	7.60%
texture	0.0833	0.0833	1	8.33%
synthetic	0.0833	0.0833	1	8.33%
noise	0.0850	0.0833	1	8.50%

The agreement (0.0833 vs. 0.0834 on the cameraman) is a nice demonstration that you understand where the error comes from. The astronaut image is lower because 11% of its pixels are saturated (exactly 0 or 255): a reconstruction that overshoots past the end of the range is clamped back, which hides the error. Measured separately, the MSE is 0.012 on those pixels and 0.084 on all the others, matching the prediction.

9.4 Floating-point precision of the transform

Is `double` overkill? We apply forward and inverse DCT to every tile of the cameraman *without quantization* and measure how far we land from the start:

arithmetic	max abs error	rms error
float64	2.7e-13	4.8e-14
float32	6.1e-05	1.0e-05
fixed-point constants, 8 fractional bits	2.7e+00	1.6e+00
fixed-point constants, 10 fractional bits	2.7e-01	3.5e-02
fixed-point constants, 12 fractional bits	1.3e-01	3.1e-02
fixed-point constants, 13 fractional bits	5.4e-02	3.1e-02
fixed-point constants, 16 fractional bits	1.0e-02	5.9e-03

- **float64** (`double`): errors of 10^{-13}, utterly irrelevant.
- **float32**: 10^{-4} worst case. Still 3 orders of magnitude below the 0.5 that would flip a pixel, so a `float` implementation is perfectly adequate for this codec (and is what GPUs prefer).

- **Fixed-point constants** with b fractional bits: the matrix is no longer exactly orthonormal, so forward and inverse do not cancel. About 10 bits are needed to keep the worst case under half a grey level, and 13 bits get below 0.1, which is why libjpeg's accurate integer DCT uses 13-bit constants. *Caveat:* this experiment only rounds the constants; a real integer implementation also rounds intermediate sums, which adds more error.

9.5 Rounding ties: why NumPy and C++ disagreed

The surprise. When I first cross-checked the C++ codec against the NumPy reference, 13 of 16 384 coefficients differed. All 13 were within 1.5×10^{-14} of an exact $x.5$.

Why ties are common. The DCT basis values at positions $(u, v) \in \{0, 4\}^2$ are rational (because $\cos^2(\pi/4) = \frac{1}{2}$), so those 4 coefficients per tile are exact multiples of $1/8$. At quality 100 one in eight of them lands exactly on $x.5$: $\frac{4}{64} \cdot \frac{1}{8} = 0.78\%$ of all coefficients, matching the measured 0.78%:

quality	coefficients	within 10^{-9} of a tie	share
100	262 144	2 033	0.78%
90	262 144	317	0.12%
50	262 144	55	0.02%

The fix. Round with $\lfloor x + 0.5 + 10^{-9} \rfloor$ in both implementations (the same constant is used in the decoder's final rounding). 10^{-9} is about 10^5 times bigger than the accumulated floating-point noise of an 8-point DCT ($\approx 10^{-14}$) and 10^9 times smaller than the smallest quantization step (1), so it cannot change a real decision, only the tie. After the fix, C++ and NumPy agree on every coefficient, every file byte and every decoded pixel (Chapter 10).

In plain words

Why this is worth knowing for a GPU/library role. Any time the "same" algorithm runs on two platforms (CPU vs GPU, scalar vs SIMD, FMA vs no FMA), summation order changes the last bits, and discrete decisions like "round" or "compare" can flip. Libraries therefore need either a defined tie-breaking rule or tests that allow ± 1 differences. Knowing the difference between "bit-exact" and "numerically equivalent" is a mark of experience.

Interview soundbite

"I predicted three things analytically and checked them: the coefficient range from Parseval, the quality-100 error from a Gaussian model of coefficient rounding, and the effect of float32 or fixed-point arithmetic on the transform. The most useful finding was that exact rounding ties are common at high quality because four DCT positions are multiples of 1/8, and that floating-point summation order decides them unless the tie-break is made explicit."

Chapter 10

How we know it is correct

Resume line: "Developed corresponding decompression routines and validated reconstructed images against original inputs."

A compression codec is dangerous to get wrong silently: a bug may still produce a plausible-looking picture. So correctness is checked in four independent layers, each catching different mistakes.

Layer	What it checks	Catches
1. Unit tests (C++, 28 groups, 32 581 checks)	Each stage in isolation against an independent definition or an exact property.	A wrong table entry, off-by-one in zigzag, sign error in amplitude bits, broken canonical code.
2. Round trips on real and random images	Encode then decode: dimensions, exact entropy-stage losslessness, bounded error at quality 100, monotonic quality.	Mistakes that only appear on odd sizes (1×1, 7×9), flat images, extreme values.
3. Independent NumPy model	Same algorithm re-implemented with different code; compare coefficients, *bytes* and pixels.	A mistake shared by the tests and the C++ (same misunderstanding). A second implementation rarely makes the same error.
4. libjpeg comparison	Same quality $\Rightarrow$ same PSNR; similar size.	A transform or table that is self-consistent but not the real JPEG algorithm.

10.1 Layer 1: what the unit tests prove

Test	Property
`dct_matches_textbook_definition`	Fast matrix DCT equals the 4-fold sum on 50 random tiles ($< 10^{-10}$).
`dct_inverse_recovers_input`	Forward then inverse returns the tile ($< 10^{-10}$).
`dct_matrix_is_orthonormal`	$CC^\top = I$ ($< 10^{-12}$).
`dct_preserves_energy_parseval`	Sum of squares unchanged by the DCT.
`dct_flat_block_...`	Flat tile $\Rightarrow$ only DC, equal to $8\times$ value.
`zigzag_is_permutation_...`	0..63 each exactly once; first 16 match JPEG.
`quant_table_scaling_rules`	Quality 50 = base table, 100 = all ones, 1 = all 255, monotone in between, clamped input.
`quantize_rounds_half_up`	Ties go up; real differences are not swallowed by the ε.
`category_and_amplitude_...`	Round trip for all values in $[-2047, 2047]$.
`tokenizer_emits_zrl_and_eob_...`	A run of 18 zeros becomes ZRL + (run 2).
`tokenizer_omits_eob_...`	No EOB when coefficient 63 is non-zero.

Test	Property
`tokenizer_dc_is_differential`	DC is coded as difference to the previous tile.
`huffman_textbook_example`	Lengths 1,3,3,3,4,4 as in the textbook.
`huffman_kraft_...`	On random distributions: $\sum 2^{-\ell} = 1$, prefix-free, $H \le L < H+1$.
`huffman_edge_cases`	Empty, single, and two-symbol alphabets.
`huffman_length_limit_is_enforced`	45 Fibonacci frequencies stay ≤ 32 bits and still round-trip.
`huffman_table_serialization_...`	Serialize then deserialize gives identical codes.
`bitio_random_round_trip`	5000 random-width bit fields (0–32 bits).
`entropy_stage_is_exactly_lossless`	Decoded coefficient array equals the encoder's, bit for bit, over 20 image/quality combinations.
`round_trip_keeps_dimensions_...`	Nine sizes including 1×1, 1×20, 257×129.
`flat_images_survive_exactly_...`	Constant images at quality 100 return exactly.
`quality_100_error_is_tiny_...`	Max error ≤ 3, MSE < 0.5.
`quality_trades_size_for_fidelity_...`	As quality rises, size and PSNR never decrease.
`extreme_dc_jumps_round_trip`	Black/white tile checkerboard (largest DC differences).
`fused_and_staged_paths_...`	Fast and profile-friendly code paths are bit-identical.
`encoding_is_deterministic`	Same input $\Rightarrow$ same bytes.
`coefficient_range_...`	$\lvert\text{coefficient}\rvert \le 1024$.
`malformed_streams_...`	Bad magic and truncations throw; 200 random bit-flips never crash.

In plain words

Property-based thinking. Many tests do not compare to a stored "expected output". They check a *property that must hold for every input* ("decode(encode(x)) has the same size as x", "code lengths satisfy Kraft equality") on randomly generated inputs. This finds bugs that hand-picked examples miss, and is the kind of testing used for numerical libraries.

10.2 Layer 3: the NumPy reference model

`python/jlc_ref.py` re-implements the codec independently. The transform stages are NumPy one-liners over all tiles at once (`C @ blocks @ C.T`); the entropy stages are plain Python loops. It reads and writes the same `JLC1` format. `python/crosscheck.py` then runs 8 images (the photo, noise, gradient, a synthetic picture, a 37×53 image, a 1×1 image, a flat image and a black/white block image) at 5 qualities each and verifies four things per case:

1. C++-decoded coefficients = NumPy analysis,
2. C++ file = NumPy file, **byte for byte**,
3. NumPy decodes the C++ file to the same pixels as the C++ decoder,
4. C++ decodes the NumPy file to the same pixels.

Result: **160 comparisons, 0 mismatches.** A full-size run on the four 512×512 images at two qualities is

also byte-identical. Getting there exposed the rounding-tie issue of Chapter 9: the very first run had 15 mismatches. That is precisely what a reference model is for.

Interview soundbite

"I validated the codec in four layers: unit tests on each stage, round-trip and property tests, an independent NumPy implementation that produces byte-identical files, and a comparison with libjpeg showing identical PSNR at equal quality. The NumPy cross-check found a real numerical issue, rounding ties decided by floating-point summation order, which I fixed with a deterministic tie-break."

Chapter 11

Performance: where the time goes

Resume line: "Benchmarked encoding and decoding performance and profiled computationally intensive stages of the pipeline."

Read this first

The numbers in this chapter were measured in the *sandbox used to prepare this guide* (a 2-core virtual machine, Intel Xeon @ 2.1 GHz). They are **not bwUniCluster results**. They illustrate the method and the kinds of findings to look for. For the interview, run the benchmarks on bwUniCluster (Chapter 13) and quote *those* numbers, with the node type, compiler and flags.

11.1 How the code measures itself

`encode()` and `decode()` take an optional timings struct. Inside, a steady-clock timer wraps each stage (in the order of the pipeline). The benchmark program `jlc_bench`:

- runs one *warm-up* iteration that is thrown away (first-touch page faults, instruction cache, frequency ramp-up),
- runs the configuration several times and reports the **median** of each stage (robust to a noisy neighbour; the mean is not),
- reports throughput in megapixels per second, and writes CSV for plotting.

11.2 The profile

Encode and decode of a 2048×2048 synthetic image at quality 50 ("staged" mode, every stage a separate pass over the whole image):

stage (sandbox, 2048×2048, quality 50)	ms
DCT	41.6
quantize + zig-zag	14.3
DC prediction + run-length tokens	6.8
build Huffman tables	0.02
bit packing	3.3
Huffman decode	9.4
dequantize + de-zig-zag	22.2
inverse DCT + round	23.3
crop	1.03

Two things stand out.

1. The DCT dominates the encoder (about 60% of encode time). That is expected: 1024 multiply-adds per tile and $2048^2/64 = 65{,}536$ tiles make 67 million multiply-adds per transform pass. Entropy coding

(tokenize + Huffman + bit packing) is much cheaper at quality 50, because most coefficients are zero; at quality 90 it grows, since there are more non-zero values to code.

2. The "dequantize" stage is as slow as the whole Huffman decoder (22 ms against 9 ms), even though it does almost no arithmetic: one multiply per coefficient. A stage with that little maths should be near-free. This is a smell: the cost is not computation, it is *memory*.

11.3 Profile → hypothesis → experiment → fix

Hypothesis. The staged decoder allocates a full-size array of `double` (8 bytes per pixel, 32 MiB at 2048^2) for the dequantized coefficients. As I understand glibc, `malloc` adapts its `mmap` threshold upwards when large blocks are freed, but only up to 32 MiB; a request of 32 MiB or more is always served with fresh zeroed pages from the kernel, and the first write to each 4 KiB page triggers a page fault. The dequantize loop would then pay for 8192 page faults per call. (The experiment below, not this allocator detail, is the evidence.)

Experiment. Run the identical benchmark with `malloc` configured to never use `mmap` and never give memory back (environment variables `MALLOC_MMAP_THRESHOLD_`, `MALLOC_TRIM_THRESHOLD_`, `MALLOC_TOP_PAD_`). Pages then stay mapped and are reused between iterations. Script: `python/page_fault_experiment.py`.

configuration (sandbox, 2048×2048, q 50)	encode [ms]	decode [ms]	dequantize stage [ms]
staged, default malloc	69.6	58.8	23.1
staged, mmap disabled	51.9	41.9	8.9
fused, default malloc	47.7	37.7	0.0
fused, mmap disabled	49.0	38.1	0.0

Result. With mmap disabled the staged decoder's dequantize stage drops from about 23 ms to 9 ms and the whole decode by about 17 ms, while the fused path does not change at all. The hypothesis is confirmed: roughly 14 ms of the 23 ms was the operating system handing out fresh pages, not arithmetic.

Fix (in the code). The *fused* execution strategy: instead of three whole-image passes with two big intermediate arrays, do DCT→quantize (encode) or dequantize→IDCT (decode) tile by tile in a 64-element local buffer that lives in L1 cache. No large allocation, no extra passes over memory. The output is bit-identical to the staged path (test `fused_and_staged_paths_are_bit_identical`), and it is selected by a flag, so the staged mode stays available for per-stage profiling.

	encode [ms]			decode [ms]		
image (q 50)	staged	fused	speed-up	staged	fused	speed-up
512×512	3.1	2.9	1.07×	2.4	2.2	1.09×
1024×1024	12.4	12.9	0.96×	9.1	9.4	0.97×
2048×2048	67.8	51.3	1.32×	57.5	42.8	1.34×
4096×4096	287.9	206.6	1.39×	256.0	193.6	1.32×

Below 2048^2 there is no gain here, consistent with allocations smaller than 32 MiB being recycled by `malloc`; from 2048^2 upwards fusing gives about 1.3–1.4×. **This size-dependence is specific to glibc and this machine**; on bwUniCluster the threshold or the kernel's page-fault cost may differ, so measure, do not assume.

Interview soundbite

"Profiling the staged decoder showed a stage that does almost no arithmetic taking as long as Huffman decoding. I hypothesized it was page-fault cost from allocating a 32 MiB temporary, confirmed it by changing only the allocator behaviour, and then removed the temporary by fusing dequantization and IDCT per tile. The output is bit-identical, verified by a test, and about 1.3× faster on large images."

In plain words

Why this matters for GPU and library work. Allocation in a hot path is a classic performance bug: it is why CUDA libraries ask you to pass in a pre-allocated workspace and why `cudaMalloc` is never called per frame. The same "allocate once, reuse" lesson applies on CPUs.

11.4 How to profile on the cluster

1. **Stage timers** (built in): run `slurm/02_bench.sbatch`, plot with `python/plot_bench.py`. This tells you *which stage* is slow.
2. **gprof** (always available with GCC): `slurm/03_profile.sbatch` builds with `-pg -fno-inline` and produces a function-level table (`results/gprof_flat.txt`). Because inlining is disabled, small helpers show up as separate rows; the *ranking* of the big functions is what you read, not the absolute time.
3. **perf** (if installed on the node; check with `which perf`): `perf stat -e cycles,instructions,cache-misses ./build/jlc_bench ...` gives instructions per cycle and cache miss counts. If it is not available, say so and rely on the first two methods; do not invent hardware-counter numbers.

11.5 Fill in your cluster results

partition / CPU	encode [Mpix/s]	decode [Mpix/s]	slowest stage	fused vs staged
`cpu_il` Xeon Platinum 8358				
`cpu` AMD EPYC 9454				

(Use the 2048^2, quality-50 rows of `results/bench_*.csv`; `python/plot_bench.py` writes `bench_summary.md` with every number.)

11.6 How I would make it faster (and what to say about each idea)

Idea	Why it helps	Caveat
Fused tile loop (done)	Removes full-image temporaries and page faults.	Hides per-stage timing, so keep staged mode.
Fast DCT algorithm (e.g. AAN)	About 5 multiplies + 29 adds per 8-point transform instead of 64 multiply-adds.	Scale factors must be folded into the quantization table; changes rounding slightly, so bit-exactness vs. NumPy would be lost.
SIMD across tiles (AVX2)	Do 8 tiles at once: each SIMD lane holds the same coefficient of a different tile. You know this technique from project 1.	Needs a transposed tile layout.
OpenMP over tile rows	DCT, quantization, and the whole decoder after Huffman are independent per tile. Near-linear speed-up until memory bandwidth saturates.	Tokenization has a DC-prediction dependency, and bit packing depends on all previous tiles' lengths.
Table-driven Huffman decode	Look up 8–12 bits at once instead of one bit per loop iteration. The bit-by-bit decoder is the clear next bottleneck at high quality.	Needs a 2-level table for long codes.
Restart intervals	Reset DC prediction and byte-align every N tiles so each segment can be decoded independently. This is how real JPEG decoders parallelize the serial Huffman stage.	Costs a few bits per interval.

11.7 The GPU connection (relevant to the NVIDIA role)

JPEG is a textbook case of a pipeline whose stages have different parallel structure:

- **DCT, quantization, dequantization, IDCT: embarrassingly parallel.** Every tile is independent; map tiles to threads or thread-blocks, with the 64 values of a tile in shared memory. This is where a GPU gives a huge speed-up.
- **Huffman decoding is inherently serial.** Where tile n starts in the bit stream depends on the code lengths of tiles $0..n-1$, and codes have variable length, so you cannot jump into the middle. Approaches: restart intervals (independent segments, a property of the *file*), or parallel schemes that decode speculatively from several bit offsets and then synchronize (an active research topic). As far as I know, NVIDIA's `nvJPEG` library therefore offers several backends that split the work differently between CPU and GPU (and a dedicated hardware engine on some data-centre GPUs); check its documentation for the current list before quoting details.
- **Memory behaviour dominates.** Tile-by-tile fused kernels, coalesced loads, and pre-allocated workspaces are the GPU analogue of what the fused path does on the CPU.

This is a good thing to discuss: it shows you understand *why* one pipeline stage is the bottleneck for parallelization, not just how to code it.

Chapter 12

Running everything on your own machine

You need a C++17 compiler (GCC or Clang), `make`, and, for the analysis scripts, Python 3 with NumPy, Pillow, matplotlib and scikit-image (`pip install numpy pillow matplotlib scikit-image`). Test images are generated locally, no download needed.

12.1 Build and test (2 minutes)

```
cd jpeg_codec
make                      # builds build/libjlc.a, build/jlc, build/jlc_bench, build/jlc_tests
make test                 # 28 test groups, 32,581 checks, must end with "0 failures"
python3 python/make_test_images.py     # writes data/*.pgm (cameraman, astronaut, noise, ...)
```

12.2 Use the codec

```
./build/jlc encode data/camera.pgm out.jlc -q 50     # compress
./build/jlc decode out.jlc out.pgm                    # decompress
./build/jlc eval   data/camera.pgm -q 50              # round trip, prints bytes, bpp, ratio, MSE,
    PSNR
./build/jlc sweep  data/camera.pgm                    # CSV over many qualities
python3 python/make_test_images.py --convert myphoto.png myphoto.pgm   # use your own picture
```

12.3 Reproduce every result in this guide

```
python3 python/crosscheck.py            # C++ vs NumPy: expect "0 mismatches"
python3 python/rd_sweep.py              # rate-distortion curves + libjpeg comparison -> results/
python3 python/numerics_study.py        # rounding / range / precision study -> results/numerics.md
python3 python/page_fault_experiment.py # the malloc / page-fault experiment (Chapter 11)
./build/jlc_bench --sizes 512,1024,2048 --quality 10,50,90 --reps 7 --csv results/bench_local.csv
python3 python/plot_bench.py results/bench_local.csv
```

12.4 Rebuild this PDF

```
python3 python/make_figures.py && python3 python/extract_code.py   # regenerate figures, numbers,
    code excerpts
cd docs && pdflatex guide.tex && pdflatex guide.tex                # twice, for the table of
    contents
```

12.5 Exercises that make the project yours

Predict the result *before* you run each change; being able to predict is what interviewers probe.

1. **Replace the zig-zag with a plain row-by-row scan.** Predict the change in file size at quality 50. (Expect larger files: the zeros no longer bunch up, so there are more and shorter runs. Measured with the NumPy model on the cameraman: +9.1%, 22 997 instead of 21 079 bytes.)
2. **Send the raw DC value instead of the difference to the previous tile.** Predict the direction and rough size from Figure 5.2 (entropy 6.0 vs 4.1 bits per tile). Measured: +6.3%, i.e. 2.6 extra bits per tile. It is not exactly the 1.9-bit entropy gap, partly because the codec Huffman-codes only the category and sends the amplitude bits raw.
3. **Set the tie epsilon to 0 and run `crosscheck.py`.** Predict which qualities mismatch. (Mostly the high ones; Chapter 9.)
4. **Use `float` instead of `double` in the DCT.** Predict whether the output changes at quality 50 (rarely, only on ties) and at quality 100.
5. **Change tile size to 16×16.** (Needs a new DCT matrix and zig-zag.) Predict the effect on compression ratio and on blocking artifacts.
6. **Parallelize `analyze()` with `#pragma omp parallel for` over tile rows.** Verify the output is unchanged, then measure speed-up on 1, 2, 4, 8 cores on the cluster.
7. **Implement a 10-bit lookup table for Huffman decoding.** Verify the output is unchanged, measure the decode speed-up at quality 90.
8. **Add colour:** convert RGB to YCbCr, code Y as now, subsample Cb/Cr by 2 in each direction, and use a separate (coarser) quantization table for chroma.

Chapter 13

Running on bwUniCluster 3.0: step by step

bwUniCluster 3.0 is the shared HPC system operated by KIT for universities in Baden-Württemberg, including Freiburg. You use it by logging in to a *login node*, preparing your work there, and handing the real computation to a job scheduler called **Slurm**, which runs it on a *compute node*.

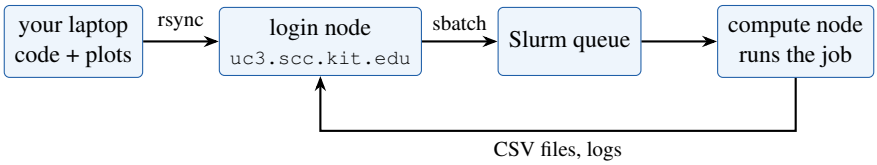
In plain words

Mental model. The login node is a shared reception desk: fine for editing files and submitting forms, *not* for computing (heavy computations on login nodes are not allowed). The compute nodes are the workshops. A *job script* is the form you hand in: "give me 1 core for 30 minutes on an Intel node, then run these commands". Slurm queues the form and runs it when a node is free.

Facts and where they come from

Queue names, limits and hardware below come from the official bwHPC wiki pages listed at the end of this chapter, which I read while preparing this guide. HPC sites change these things. Before relying on a number, confirm it on the cluster with `sinfo -s` (queues), `module avail` (software) and the wiki.

13.1 The workflow in one picture



13.2 One-time setup: account and login

1. **Registration.** The login documentation lists three prerequisites: complete the 3-step registration procedure, set a *service password* for bwUniCluster 3.0, and set up a *second factor* (a TOTP app on your phone). You enter the one-time code first and then the service password when you log in. Since you already ran jobs on the cluster for project 1, this is probably done; if the login fails, this is the first thing to re-check.
2. **Network.** Access is limited to the BelWü research network. On the Freiburg campus network you are normally inside it; from home use the university VPN or an SSH jump host. A connection that times out from outside is not a bug in your command.
3. **Log in.**

```
ssh <your_username>@uc3.scc.kit.edu
```

Your username is a university prefix plus your local name (for KIT users, for example, `ka_ab1234`); the registration pages show yours. Alternative host names listed on the wiki: `bwunicluster.scc.kit.edu`, `uc3-login1.scc.kit.edu`, `uc3-login2.scc.kit.edu`.

13.3 Where to put files

`$HOME`	Permanent, backed up, same on all nodes. Put source code and job scripts here. Our project is a few megabytes, so it lives here.
Workspaces	For large temporary data (restart files, big outputs). Created with `ws_allocate <name> <days>`, found with `ws_find <name>`, listed with `ws_list`, extended with `ws_extend <name> <days>`; they expire. You do not need one for this project unless you benchmark with many gigabytes of images.
`$TMPDIR`	Fast node-local disk, exists only while a job runs. Irrelevant here.

13.4 Step 1: copy the project to the cluster

From your laptop (in the folder that contains `jpeg_codec/`):

```
rsync -av --exclude build --exclude results/sandbox_example jpeg_codec/ <user>@uc3.scc.kit.edu:~/
    jpeg_codec/
```

(Or push to a private GitHub repository from your laptop and `git clone` on the cluster.) Then on the cluster:

```
cd ~/jpeg_codec
mkdir -p logs results data
```

The test images are produced by a Python script. If NumPy and scikit-image are not available on the login node, generate `data/*.pgm` on your laptop (`python3 python/make_test_images.py`) and include the `data/` folder in the rsync. The C++ benchmark with synthetic images needs no data files at all.

13.5 Step 2: look around

```
module avail compiler/gnu        # which GNU compiler versions exist
module spider gnu                # (Lmod) also finds modules with unmet dependencies
sinfo -s                         # summary of queues (partitions) and node counts
squeue --me                      # your own jobs
```

The cluster uses **Lmod**: `module load`, `module list`, `module purge` and `module spider` are the commands you need. The wiki lists GNU compilers `compiler/gnu/11.4` and `compiler/gnu/14.2` (`13.3` is also referenced). The file `slurm/env.sh` tries `compiler/gnu/14.2` first and falls back to the default `compiler/gnu`; edit it if `module avail` shows different versions. The project needs C++17 (it uses structured bindings). I built and tested it with GCC 13.3; any recent GCC that supports C++17 should work, but I have not tried older versions, so if the build fails on an old compiler module, load a newer one.

13.6 The queues you will use

queue	node type	cores / memory	max wall time	use it for
`dev_cpu`	development queue	as `cpu`	30 min	compile, test, short checks
`cpu_il`	Intel Xeon Platinum 8358 (Ice Lake)	64 cores, 256 GiB	72 h	benchmark
`cpu`	AMD EPYC 9454	96 cores, 384 GiB	72 h	benchmark
`highmem`	AMD EPYC 9454	96 cores, 2.3 TiB	72 h	not needed

Development queues (`dev_` prefix) have a 30-minute limit, allow only *one running job at a time* and at

most 3 queued, and are meant for "developing, compiling and testing code". The wiki explicitly says not to misuse them for regular short-running production jobs or chained jobs. So: build and test in `dev_cpu`; run the real benchmark in `cpu_il` and `cpu`.

13.7 Step 3: build and test on a compute node

```
sbatch slurm/01_build_test.sbatch
squeue --me                          # PD = pending, R = running. Check again after a minute or two.
cat logs/jlc-build_<jobid>.out       # expect: "32581 checks, 0 failures"  and "build + tests OK"
```

Be kind to the scheduler

The wiki asks you not to run `squeue` or other Slurm commands in loops or with `watch`, and to leave at least 30 seconds between queries. Submit, wait, check.

If you prefer an interactive session (type commands yourself on a compute node), the wiki's example is:

```
salloc -p dev_cpu -n 1 -t 20 --mem=4000      # asks for 1 task, 20 minutes, 4000 MB; you land on
    the node
module load compiler/gnu && make && make test
exit                                          # give the node back!
```

13.8 Step 4: the benchmark jobs

```
bash slurm/submit_all.sh          # submits slurm/02_bench.sbatch to cpu_il and to cpu
squeue --me
```

What each job does (read `slurm/02_bench.sbatch`; every line is commented):

1. loads the compiler, prints `module list` and `g++ -version` (it ends up in the log),
2. writes `results/meta_<partition>.txt`: date, host name, CPU model (`lscpu`), cache sizes, compiler. **Keep these files**: a benchmark number without the machine it ran on is worthless,
3. runs `jlc_bench` on synthetic images of $512^2, \ldots, 4096^2$ pixels at quality 10, 50, 90, both *staged* and *fused*, with 7 repetitions each (median reported),
4. runs it on the real photographs, 15 repetitions,
5. writes `results/bench_<partition>_fused0.csv`, `..._fused1.csv`, and one CSV per photograph.

13.8.1 What the #SBATCH lines mean

`-partition=cpu_il`	Which queue (node type). Overridden by `sbatch -partition=cpu ...`.
`-nodes=1 -ntasks=1`	One node, one process.
`-cpus-per-task=1`	One core: the codec is single-threaded, so one core is the honest request.
`-mem=8000mb`	8000 MB of memory. The largest image uses well under 1 GB; the default per-core memory on these queues is about 1–2 GB, so ask explicitly.
`-time=00:30:00`	Wall-time limit. The job is killed at the limit, so be generous but not absurd (short jobs are scheduled sooner).
`-output=logs/%x_%j.out`	Log file: `%x` = job name, `%j` = job number. **The directory must already exist** (`mkdir -p logs`) or the job fails immediately.
`srun -cpu-bind=cores`	Starts the program on the allocated core and pins it there, so it cannot migrate between cores during the measurement.

13.8.2 Benchmark hygiene (what makes a number trustworthy)

- **Warm-up run discarded**, then several repetitions, **median** reported (built into `jlc_bench`).
- **Same binary, same input, same settings** on both machines; only the node changes. Compare partitions only with this in mind: different CPUs, different clocks, different caches.
- **The node may be shared.** Other users' jobs can run on the same node and disturb timings (memory bandwidth, shared caches). For the cleanest numbers add `-exclusive` to the `sbatch` command (you wait longer in the queue and use the whole node for 30 minutes). Alternatively repeat the job at another time and check the medians agree within a few per cent.
- **Report what you measured**: "median of 7 runs, single core, GCC ..., `-O3`, node type ...".
- **Optional extra:** `NATIVE=1 sbatch -partition=cpu_il slurm/02_bench.sbatch` also rebuilds with `-march=native` on the compute node and benchmarks that binary separately. The compiler may then auto-vectorize the DCT loops with AVX2/AVX-512; compare with the portable build. A native binary can crash with "illegal instruction" on a different CPU type, so the script names it per partition.

13.9 Step 5: profile

```
sbatch slurm/03_profile.sbatch       # dev_cpu, ~1 minute
less results/gprof_flat.txt          # function-level table (top rows = hot spots)
```

13.10 Step 6: bring the results home and plot

On your laptop:

```
rsync -av <user>@uc3.scc.kit.edu:~/jpeg_codec/results/ ./cluster_results/
python3 python/plot_bench.py cluster_results/bench_cpu_il_fused0.csv cluster_results/
    bench_cpu_fused0.csv --outdir cluster_results
```

This creates `bench_stages.png` (where the time goes, one bar per machine), `bench_scaling.png` (throughput versus image size) and `bench_summary.md` (all numbers as tables, ready to paste into notes). Fill in the table at the end of Chapter 11.

13.11 When something goes wrong

Symptom	Likely cause and fix
`ssh` hangs or is refused	Not in the BelWü network: connect to the university VPN. Wrong code order: one-time code first, then service password.
`module: command not found` in a script	The script is not run through a login shell. `slurm/env.sh` checks for it; check `module list` at the top of the log.
Module not found	Version differs: `module avail compiler/gnu`, `module spider gnu`, then edit `slurm/env.sh`.
Job stays `PD` (pending)	Normal. `squeue -me` shows a reason: `Priority`/`Resources` = waiting for a free node; for `dev_cpu` you may only have one running job.
Job fails in 1 second, empty log	`logs/` directory missing, or a typo in `-partition`. Look at `scontrol show job <id>` and run `sinfo -s`.
`Killed` / out-of-memory	Raise `-mem`. (4096^2 needs well under 1 GB.)
"Illegal instruction"	A `-march=native` binary was run on a different CPU type; rebuild on the node that runs it.
Numbers differ a lot between runs	Shared node. Use `-exclusive`, more repetitions, or compare medians from separate runs.
Tests fail on the cluster but not locally	Should not happen (the results are deterministic). Check `g++ -version` and that you rsynced the up-to-date sources; then investigate: it would be a real finding.

13.12 How to describe it truthfully

After you have run the jobs, a precise sentence you can use in the interview and on the resume is, for example:

> "I benchmarked the codec on bwUniCluster 3.0 nodes (Intel Xeon Platinum 8358 and AMD EPYC 9454) with GCC and `-O3`, single core, median of 7 runs, and profiled the stages with gprof; *[your measured numbers here]*."

Only claim what the job logs and `meta_*.txt` files show. If something did not run (for example `perf` is not installed), say so; it is a normal answer.

13.13 Sources

All retrieved while preparing this guide; re-check them, since clusters change.

- Batch queues: `https://wiki.bwhpc.de/e/BwUniCluster3.0/Batch_Queues`
- Running jobs (`sbatch`, `salloc`, monitoring): `https://wiki.bwhpc.de/e/BwUniCluster3.0/Running_Jobs`
- Software modules (Lmod): `https://wiki.bwhpc.de/e/BwUniCluster3.0/Software_Modules`
- Login, registration, network restriction: `https://wiki.bwhpc.de/e/BwUniCluster3.0/Login`
- Hardware and file systems: `https://wiki.bwhpc.de/e/BwUniCluster3.0/Hardware_and_Architecture`
- Workspaces: `https://wiki.bwhpc.de/e/Workspaces`

Chapter 14

The resume bullets, one by one

For each bullet: what it means in plain words, how it was done, where to find it, why it was done that way, what evidence you can point to, and the follow-up question to expect.

14.1 Bullet 1

"Implemented a compact image compression pipeline using 8x8 DCT, quantization, zig-zag scanning, run-length encoding, and Huffman coding."

What A complete encoder for grayscale images following baseline JPEG's design, with its own file format.

How Matrix-form 8×8 DCT ($CXC^\top$); libjpeg-style quality-scaled luminance quantization table; zig-zag scan generated from anti-diagonals; DC differences plus (run, size) tokens with ZRL and EOB; two canonical Huffman tables built per image.

Where `src/dct.cpp`, `quant.cpp`, `zigzag.cpp`, `rle.cpp`, `huffman.cpp`, `codec.cpp` (Chapters 2–7).

Why these choices DCT for energy compaction; tile size 8 as in JPEG; per-image Huffman for the smallest files (a second pass is cheap); canonical codes so only lengths are stored.

Evidence 28 test groups; byte-identical NumPy model; PSNR equal to libjpeg at equal quality.

Expect "Why not just use libjpeg?" (Because the goal was to understand and measure each stage; libjpeg was used as the independent reference.)

14.2 Bullet 2

"Developed corresponding decompression routines and validated reconstructed images against original inputs."

What A decoder that is the exact inverse chain, plus checks that its output matches the input within the error that quantization allows.

How Parse header and tables, Huffman-decode tiles, dequantize, inverse DCT, level-shift and clamp, crop. Validation is layered: exact losslessness of the entropy stage, bounded error at quality 100, monotone behaviour with quality, round trip on odd sizes, robustness to damaged files.

Where `decode()` in `src/codec.cpp`; `tests/test_main.cpp`; Chapter 10.

Evidence `entropy_stage_is_exactly_lossless`, `quality_100_error_is_tiny_and_bounded`, the 160-case cross-check.

Expect "What does "validated" mean for a lossy codec?" (You cannot demand equality, so you test properties: exactness where the math is lossless, tight error bounds where it is not, and agreement with an independent implementation.)

14.3 Bullet 3

"Analyzed the rate-distortion trade-off using compression ratio, MSE, and PSNR across quantization levels."

What Swept the quality setting from 1 to 100 on six images and plotted rate against distortion.

How `jlc sweep` for each image; `python/rd_sweep.py` adds the libjpeg curves, plots, and tables.

Where `results/rd_*.csv`, `rd_curves.png`; Chapter 8.

Key numbers
Cameraman: 12.4:1 and 32.6 dB at quality 50 (0.64 bpp); 1.8:1 and 58.9 dB at quality 100; ∼130:1 at quality 1. Noise at quality 100 expands the file (ratio 0.95).

Expect "Why does PSNR plateau/explode at quality 100?" (Step size 1: only rounding error remains; MSE ≈ 0.083.)

14.4 Bullet 4

"Investigated rounding, coefficient range, and numerical reconstruction errors introduced during compression."

What Predicted and measured where numerical error comes from.

How Parseval bound ($|\text{coef}| \leq 1024$); uniform-error model $Q^2/12$ against measurement; Gaussian model of the quality-100 error ($P = 0.0833$); float32 and fixed-point precision experiments; analysis of exact rounding ties and the 10^{-9} tie-break.

Where `python/numerics_study.py`, `include/jlc/quant.hpp`; Chapter 9.

Expect "Is double precision necessary?" (No: float32 has 10^{-4} error against a 0.5 threshold. Ties are the only place where precision matters, and only for reproducibility.)

14.5 Bullet 5

"Benchmarked encoding and decoding performance and profiled computationally intensive stages of the pipeline."

What Per-stage timing, throughput vs. image size, hot-spot analysis, and one optimization driven by the profile.

How Stage timers inside `encode`/`decode`; `jlc_bench` (warm-up, median, CSV); gprof; the page-fault experiment; fused execution path.

Where `apps/jlc_bench.cpp`, `slurm/`, `python/plot_bench.py`; Chapter 11, Chapter 13.

Evidence to prepare
Your cluster numbers: Mpixel/s per partition, stage breakdown, fused vs. staged.

Expect "What dominates, and what would you do next?" (DCT on the encoder; Huffman decoding and memory traffic on the decoder; next: SIMD/OpenMP over tiles, table-driven Huffman, restart intervals.)

14.6 Resume wording to consider once your cluster numbers exist

The current bullets are accurate. If you want denser versions, use only numbers you have measured; bracketed items are yours to fill.

- Implemented a baseline-JPEG-style grayscale codec in C++17 (8×8 DCT, libjpeg-scaled quantization, zig-zag, DC DPCM + run-length, per-image canonical Huffman); a NumPy reference model produces byte-identical bitstreams.
- Validated against libjpeg: identical PSNR at equal quality and file sizes within about 1% on natural images; 28 unit-test groups (32 000+ checks) including random, odd-size and corrupted inputs.
- Characterized rate–distortion from 130:1 (quality 1) to 1.8:1 (quality 100); predicted the quality-100 reconstruction error analytically (MSE 0.083 predicted, 0.083 measured).
- Found and fixed a cross-implementation divergence caused by floating-point rounding ties (four DCT positions are exact multiples of $1/8$); added a deterministic tie-break.
- Profiled on bwUniCluster [Xeon Platinum 8358 / EPYC 9454]: [stage] dominates at [X]%; fusing dequantization and IDCT per tile removed a 32 MiB temporary and cut decode time by [Y]% with bit-identical output.

Chapter 15

Questions you should expect, with answers

These answers are written to be *spoken*: short, structured, with the numbers you will actually have. Do not memorize them word for word. Read the question, answer aloud in your own words, then compare. Every answer here is consistent with what the code does; if you change the code, change the answer.

15.1 The project

1. Walk me through the project in one minute.

"I built a JPEG-style codec for grayscale images in C++ with a NumPy reference model. The encoder cuts the image into 8×8 tiles, subtracts 128, applies an orthonormal DCT, divides by a quality-scaled quantization table and rounds, which is the only lossy step. Then it reorders each tile in zig-zag, codes the DC as a difference to the previous tile, run-length codes the AC values as (run, size) symbols, and Huffman-codes those with per-image canonical tables. The decoder inverts that. I validated it with unit tests, a byte-identical Python model and a comparison with libjpeg (same PSNR at equal quality), measured rate–distortion, studied the numerical error sources, and profiled the stages on bwUniCluster."

2. Why the DCT, and not simply quantizing the pixels?

Quantizing pixels directly throws away information everywhere equally and gives no structure to exploit. The DCT is an orthonormal change of basis that concentrates the energy of a smooth tile into a few low-frequency coefficients; the rest are near zero. Quantization then removes small coefficients almost for free, and the zeros are cheap to code. It is orthonormal, so the inverse is the transpose and squared error is preserved, which keeps distortion easy to reason about.

3. Why the DCT rather than the Fourier transform?

Real output rather than complex, and better behaviour at tile borders. The DFT assumes the tile repeats periodically, which creates an artificial jump between the right and left edge; the DCT effectively mirrors the tile, so there is no jump and the energy stays in fewer coefficients.

4. Why 8×8 tiles?

A compromise. Larger tiles capture more correlation and compress smoother areas better, but cost more arithmetic, blur edges across a bigger area, and need more memory; smaller tiles are cheaper but exploit less redundancy and make blocking more visible. 8 is JPEG's historic choice; modern codecs use variable sizes up to 64 or more, chosen per region.

5. Where exactly is information lost?

Quantization. After that everything is reversible: zig-zag, run-length and Huffman coding are exactly invertible, and my test proves the decoded coefficients equal the encoder's bit for bit. The decoder adds two tiny effects: rounding to integer pixels and clamping to $[0, 255]$.

6. Why zig-zag?

It orders coefficients from low to high frequency. High-frequency coefficients are mostly zero after quantization, so the zeros form one long run at the end that a single end-of-block symbol replaces. A

row-by-row scan interleaves zeros and non-zeros. On the cameraman, replacing zig-zag with row-major order made the file 9.1% larger.

7. Why code the DC coefficient as a difference?

Neighbouring tiles have similar average brightness, so the difference is concentrated near zero and needs a shorter code. Entropy dropped from 6.0 to 4.1 bits per tile; sending raw DC values made the file 6.3% larger in my experiment.

8. Explain Huffman coding. What is a canonical code and why use one?

Count symbol frequencies, repeatedly merge the two rarest into a parent, and read the code off the tree path; frequent symbols get short codes, and the code is prefix-free so it decodes unambiguously. Average length is within one bit of the entropy. A canonical code assigns the actual bit patterns by a fixed rule from the lengths alone (sort by length then symbol, increment, shift when the length grows), so the file stores only code lengths, and decoding becomes a small range check per length.

9. Huffman is not optimal in general. What is better?

Huffman must spend a whole number of bits per symbol, so it loses when a symbol has probability close to 1 (it cannot spend 0.1 bit). Arithmetic and range coding approach the entropy exactly; the JPEG standard defines an arithmetic-coded variant, rarely used for historic patent reasons and slower speed. For this project Huffman matches the standard baseline.

10. What does the quality parameter mean?

It scales the quantization table. Quality 50 uses the standard table, lower quality multiplies the steps (quality 10: 5×), higher quality shrinks them (quality 100: all steps 1). The rule is libjpeg's, so quality numbers are comparable with libjpeg's.

15.2 Measurement and numerics

11. What are PSNR and MSE, and what are their limits?

MSE is the average squared pixel error; PSNR $= 10\log_{10}(255^2/\text{MSE})$ in dB. Every +6 dB halves the typical error. Limit: it measures pixel distance, not perception; blur and blocking with equal MSE look different. That is why perceptual metrics such as SSIM exist. I used PSNR because it connects directly to quantization noise and I could validate it against libjpeg.

12. What did the rate–distortion curves show?

Diminishing returns: going from quality 50 to 75 on the cameraman costs 1.6× the bytes for 2.5 dB, while 95 to 100 costs 1.8× for 14 dB nobody can see. Content matters enormously: a smooth gradient compresses beyond 58:1 at quality 50, the cameraman 12:1, and white noise only 2.4:1 at 22.7 dB, and it *expands* at quality 100.

13. How did you validate against libjpeg, and what does the comparison prove?

Same images, same quality values, optimized Huffman tables via Pillow. PSNR agrees to 0.01 dB at quality 50 on all six images, which shows my DCT, tables, scaling and rounding match the standard, since distortion depends only on those. File size is within about 1% on natural images, which shows the entropy coder is competitive. It does not prove my files are valid JPEG: they are not (own container).

14. What was the most interesting bug or finding?

Cross-checking against NumPy: 13 of 16 384 coefficients differed. All were within 10^{-14} of an exact $x.5$. At four DCT positions the coefficients are exact multiples of $1/8$, so ties are common at high quality, and the rounding direction was decided by floating-point summation order. I added a deterministic tie-break ($+10^{-9}$ before the floor, justified by the noise level and the minimum step), and the two implementations became byte-identical. General lesson: when "the same algorithm" runs on different platforms, discrete decisions can flip in the last bit.

15. Is double precision needed? What about float or fixed point?

Not for accuracy. Forward plus inverse in float32 errs by 10^{-4} against the 0.5 threshold for flipping a pixel. For fixed-point constants about 10 bits keep the worst case under half a grey level and 13 bits get to 0.05; I only modelled the constants, not intermediate rounding. Double is used for simplicity and reproducibility.

16. Why is quality 100 not lossless?

Step size 1 still rounds every coefficient to an integer (error up to 0.5, variance $1/12$). Orthonormality carries that to the pixels; the final rounding hides it unless $|e| > 0.5$, which happens for about 8.3% of pixels. I predicted MSE 0.0833 and measured 0.0834 on the cameraman.

17. The tile size is not a divisor of the image size. What do you do?

Replicate the last row and column up to a multiple of 8 (a zero pad would create a fake sharp edge that costs bits), store the true size in the header, and crop after decoding. Tests cover 1×1, 7×9, 17×13 and others.

15.3 Performance and parallelism

18. Which stage dominates, and how do you know?

(Use your cluster numbers.) "On the [node type] the DCT accounts for about [X]% of encode time and Huffman decoding [Y]% of decode time at quality 90. I know this from per-stage steady-clock timers, taking the median of repeated runs after a warm-up, and from a gprof profile that ranks the same functions."

19. What optimization did you do, and how did you verify it?

In the staged decoder, the dequantize stage did almost no arithmetic but cost as much as Huffman decoding. I hypothesized page faults on a fresh 32 MiB buffer, tested it by changing only the allocator behaviour, then fused dequantization and IDCT per tile to remove the buffer. The fast path is selected by a flag and a test requires it to be bit-identical to the staged path. Gain: about $1.3\times$ on large images in my sandbox [and your cluster number].

20. How would you parallelize the codec?

DCT, quantization, dequantization and IDCT are independent per tile: OpenMP over tile rows, or SIMD across tiles (AVX2: 8 tiles at once, one coefficient per lane). Entropy coding is the hard part: DC prediction links consecutive tiles, and the bit position of each tile depends on the lengths of all previous ones. The standard answer is restart intervals, which reset DC prediction and align to a byte boundary every N tiles so the segments can be coded and decoded independently, at a cost of a few bits per interval.

21. How would this look on a GPU?

One thread-block per tile or per group of tiles, the 64 values in shared memory, row transform then column transform with coalesced loads, fused with quantization so intermediate data never goes to global memory. Huffman decoding is the serial bottleneck; the options are restart markers, speculative parallel decoding that synchronizes, or leaving that stage on the CPU and overlapping it with GPU work. As far as I know

this split is what GPU JPEG libraries such as nvJPEG expose as different backends; I would check its documentation for details rather than quote them from memory.

22. Your Huffman decoder reads one bit per iteration. How would you speed it up?

Peek at the next 8–12 bits and index a lookup table that returns the symbol and its length, with a second-level table for the rare longer codes; refill a 64-bit buffer a byte or word at a time instead of fetching bits individually.

23. What is the complexity of your DCT, and can it be faster?

Separable matrix form: $2 \cdot 8^3 = 1024$ multiply-adds per tile, versus 4096 for the direct double sum. Fast algorithms such as AAN need about 5 multiplications and 29 additions per 8-point transform (about 16 transforms per tile), with scaling folded into the quantization table. I did not use them because they change the rounding behaviour slightly, and I wanted bit-exact comparability with the NumPy model.

24. How do you make a benchmark trustworthy?

Discard a warm-up run, repeat, report the median, pin to a core, use identical binary and inputs across machines, record CPU model, compiler and flags, and be explicit that the node may be shared (use exclusive nodes or repeat on different occasions). Report throughput (Mpixel/s) as well as time so image sizes are comparable.

15.4 Scope, honesty and what next

25. How does this differ from real JPEG?

Grayscale only (no YCbCr conversion, no chroma subsampling); our own container instead of JFIF markers; one Huffman table pair per image rather than the standard tables or per-scan tables; no byte stuffing, restart markers, progressive mode or EXIF handling. The core algorithm stages and quality scaling are the baseline JPEG ones, which is why PSNR matches libjpeg.

26. How would you add colour?

Convert RGB to YCbCr, code Y as now, downsample Cb and Cr by two in each direction (the eye is less sensitive to colour detail), give chroma its own quantization and Huffman tables, and interleave the tiles. The DC prediction runs per component.

27. What causes blocking artifacts, and how are they reduced?

Tiles are quantized independently, so at low quality neighbouring tiles disagree at their borders. Mitigations: deblocking filters after decoding, overlapped/lapped transforms, larger or adaptive transforms, or wavelet codecs (JPEG 2000) that have no tile borders in the same sense.

28. What would you do next with another week?

Table-driven Huffman decode and restart intervals (to parallelize decoding), SIMD DCT across tiles, then an OpenMP strong-scaling study on one node; a perceptual metric (SSIM); and colour support. I would keep the staged and fused paths and the tie-break so that every optimization can be tested against the NumPy reference.

15.5 Beyond the project (an exploratory call)

The call is described as *exploratory*, so expect conversation about you as well. Use true facts from your own history and connect them to this project:

- **Bosch (two years):** you built Python pipelines to evaluate large measurement and simulation datasets, analyzed deviations between expected and measured behaviour, and built regression and comparison workflows. That is exactly the habit behind the reference model and cross-check: "I do not trust a result until an independent path reproduces it."
- **Embedded and HW/SW background:** fixed-point arithmetic, memory behaviour, and performance profiling are things you did before this M.Sc., so "numerical accuracy versus speed" is familiar ground.
- **Project 1 and project 2 together:** one was about making kernels fast (AVX2, OpenMP, cache-aware access); this one is about correctness, numerics and a full pipeline whose stages have different parallel structure. Together they cover the tension a library engineer lives with.

Questions you could ask them (choose two or three that fit the conversation; do not assume details about their team):

- What would a typical intern project look like in this team, and how is it validated against a reference?
- How do you decide between bit-exact and tolerance-based equivalence between CPU and GPU code paths?
- How are performance regressions detected and tracked over releases?
- Which of the image or data-processing components has the toughest trade-off between throughput and memory use right now?

Chapter 16

Drills: do these without the guide

Do each drill on paper first. The worked answers are at the end of each drill; they were computed by the same code as the project.

16.1 Hand calculations

D1. **DC of a flat row.** All 8 samples of a row equal 10. What is the 1D DCT output X_0? And what is X_1?

D2. **Category and amplitude bits.** Give size and amplitude bits for $5,\ -5,\ 12,\ -12,\ -1,\ 0$.

D3. **Tokenize.** Previous block's DC was 10. Current block in zig-zag order: $[12,0,0,3,0,0,0,-1,0,\ldots,0]$. List the tokens (symbol in hex, amplitude bits).

D4. **Huffman by hand.** Frequencies A:5, B:2, C:1, D:1. Build the code with the project's tie-break (equal frequency: smaller smallest-symbol first). Lengths, canonical codes, total bits, and bits for a fixed 2-bit code.

D5. **Decode.** With the code from D4, decode `010110011110`.

D6. **PSNR.** MSE $= 25$ and MSE $= 100$: PSNR? What happens to PSNR if the RMS error doubles?

D7. **Bits per pixel.** A 512×512 image at 0.64 bpp: how many bytes?

D8. **Quality table.** At quality 25, what are the scale s and the quantization steps for the base entries 16 and 99?

D9. **Quantize and reconstruct.** Coefficient -37.2, step 16: quantized value, reconstructed value, error, and why the error cannot exceed 8.

D10. **Zig-zag.** At which zig-zag position is the coefficient in row 1, column 2? Which (row, column) is at position 12?

16.2 Answers

D1. $X_0 = \frac{1}{\sqrt{8}} \cdot 8 \cdot 10 = 10\sqrt{8} \approx 28.28$. $X_1 = 0$ (positive and negative halves of the cosine cancel). In 2D a flat tile of value a has DC $= 8a$.

D2. 5: size 3, `101`. -5: size 3, $-5+7=2$, `010`. 12: size 4, `1100`. -12: size 4, $-12+15=3$, `0011`. -1: size 1, `0`. 0: size 0, no bits.

D3. DC difference $12-10=2$: category 2, symbol `0x02`, amplitude `10`. The first non-zero AC is 3 at position 3 with two zeros before it: run 2, size 2, symbol `0x22`, amplitude `11`. Next -1 at position 7 with three zeros before it: run 3, size 1, symbol `0x31`, amplitude `0`. All remaining are zero: EOB `0x00`.

D4. Merge C+D= 2; now B(2, smallest symbol 1) and CD(2, smallest symbol 2) are tied, so B goes first: merge B+CD= 4; then A+4= 9. Lengths A=1, B=2, C=3, D=3. Canonical codes: A=`0`, B=`10`, C=`110`, D=`111`. Total bits $5\cdot1+2\cdot2+1\cdot3+1\cdot3=15$ versus $9\cdot2=18$ for a fixed 2-bit code.

D5. `0|10|110|0|111|10` ⇒ A B C A D B.

D6. $10\log_{10}(65025/25) = 34.15\,\text{dB}$; $10\log_{10}(65025/100) = 28.13\,\text{dB}$. Doubling the RMS error quadruples MSE: $-6.02\,\text{dB}$.

D7. $0.64 \cdot 262144/8 = 20{,}972$ bytes (the cameraman at quality 50 is 21 079).

D8. $s = \lfloor 5000/25 \rfloor = 200$. Entry 16: $\lfloor (16 \cdot 200 + 50)/100 \rfloor = 32$. Entry 99: $\lfloor (99 \cdot 200 + 50)/100 \rfloor = 198$.

D9. $\lfloor -37.2/16 + 0.5 \rfloor = \lfloor -1.825 \rfloor = -2$; reconstructed $-2 \cdot 16 = -32$; error 5.2. Rounding to the nearest multiple of the step can never be off by more than half a step (8).

D10. Row 1, column 2 is natural index $1 \cdot 8 + 2 = 10$, which is at zig-zag position 7. Position 12 holds natural index 18, i.e. row 2, column 2.

16.3 Explain aloud (2 minutes each, no notes)

1. The pipeline from pixels to bytes and back, naming the one lossy step.
2. Why the inverse DCT is the transpose, and what Parseval buys you.
3. Why $\pm$ amplitude bits use the complement convention and how the decoder tells the sign.
4. Canonical Huffman: how the decoder rebuilds codes from lengths.
5. The rounding-tie bug from discovery to fix.
6. The profile, the hypothesis, the experiment, and the fix.
7. What you would parallelize on a GPU, and what you would not.

16.4 An eight-day plan

Today is Wednesday 7 October; the interview is Thursday 15 October, 11:00–12:00 CEST on Teams.

Wed 7 Oct	Build and test locally (Chapter 12); run the Python scripts; read Chapters 1–3 with the code open.
Thu 8 Oct	Chapters 4–7. Exercises 1–3 of Chapter 12: predict, then run.
Fri 9 Oct	Log in to bwUniCluster (VPN if needed), copy the project, run `01_build_test`, then *submit the benchmark jobs early*: queue waits vary. **Reply to the recruiter to confirm the interview slot** if you have not yet: the e-mail asks for it.
Sat 10 Oct	Collect results, plot, fill in the table in Chapter 11. Read Chapters 8–11. Do one optimization exercise (OpenMP or Huffman lookup table) and verify the output is unchanged.
Sun 11 Oct	Chapters 14–15 aloud; record yourself on questions 1, 5, 14, 19, 20, 21. Listen back.
Mon 12 Oct	Drills D1–D10 and the seven "explain aloud" items without notes.
Tue 13 Oct	Mock interview with a classmate for 45 minutes; give them the question list but not the answers. Fix the weak spots.
Wed 14 Oct	Light review only. Re-run `make test` so you know the project works. Check the Teams link, camera, microphone, a quiet room. Prepare two questions for them.
Thu 15 Oct	Join at 11:00. If the interviewer has not joined after 10 minutes, e-mail the recruiter, as the invitation says. No outside tools during the call.

Appendix A

Glossary and cheat sheet

A.1 Terms in plain words

AC coefficient Any DCT output other than the first: how much of a particular wiggly pattern is present.

Amplitude bits The raw bits after a Huffman code that say which value inside a category it is.

bpp Bits per pixel of the compressed file.

Canonical Huffman A Huffman code whose bit patterns are assigned by a fixed rule from the code lengths, so only the lengths need be stored.

Category / size Number of bits needed for the magnitude of a value.

Coefficient One of the 64 numbers a tile is turned into by the DCT.

DC coefficient The first DCT output: $8\times$ the tile's average brightness.

DCT Discrete Cosine Transform: re-expresses a tile as a weighted sum of 64 cosine patterns.

DPCM Sending differences to a prediction (here: previous tile's DC).

EOB End of block: "all remaining coefficients are zero".

Entropy The theoretical minimum average bits per symbol for a given distribution.

Fused / staged Two ways to run the same stages: per tile in a tiny buffer, or as full-image passes with big intermediates.

Kraft inequality $\sum 2^{-\ell_i} \leq 1$ for any prefix-free code; equality means no wasted codewords.

Level shift Subtracting 128 so pixels are centred on zero.

Lossy / lossless Whether the original can be recovered exactly.

MSE / PSNR Average squared error / its logarithmic form in dB.

Orthonormal A transform that preserves lengths and angles; inverse = transpose.

Parseval Orthonormal transforms preserve the sum of squares.

Prefix-free No code is the beginning of another.

Quality The 1–100 slider that scales the quantization table.

Quantization Dividing by a step and rounding: the lossy step.

Rate–distortion The trade-off between file size and error.

Run-length coding Describing zeros by how many there are.

Slurm The job scheduler on bwUniCluster.

Tile / block An 8×8 group of pixels.

ZRL Zero run length: “16 zeros”, used when a run exceeds 15.

Zig-zag Diagonal scan order from low to high frequency.

A.2 Command cheat sheet

```
# local
make && make test                                # build, 28 groups / 32,581 checks
python3 python/make_test_images.py               # data/*.pgm
./build/jlc eval data/camera.pgm -q 50           # one round trip with metrics
python3 python/crosscheck.py                     # C++ vs NumPy, expect 0 mismatches
python3 python/rd_sweep.py                       # rate-distortion
python3 python/numerics_study.py                 # numerics
./build/jlc_bench --sizes 512,1024,2048 --quality 10,50,90 --reps 7 --fused 0
# cluster (from the project directory on uc3.scc.kit.edu)
mkdir -p logs results data
sbatch slurm/01_build_test.sbatch                # dev_cpu: build + tests
bash slurm/submit_all.sh                         # benchmarks on cpu_il and cpu
sbatch slurm/03_profile.sbatch                   # gprof
squeue --me                                      # status; at most every 30 s
scontrol show job <id>                           # why pending / failed
scancel <id>                                     # cancel
```